FASE II Y FASE III DOCUMENTACIÓN PROYECTO S10-S11-S12

GUSTAVO ADOLFO CARDONA BAYONA

JONATHAN SOLER GUEVARA

HUGO ANDRÉS FORERO ANGEL

INGENIERÍA DE SOFTWARE, UNIVERSIDAD MANUELA BELTRÁN

SISTEMAS TRANSACCIONALES A1

JAMILTON FERNANDO BENVIDES

BOGOTÁ D.C.

16 de octubre de 2026

TABLA DE CONTENIDO

1. INFORMACIÓN GENERAL DEL PROYECTO.....3
2. OBJETIVO CENTRAL.....7
3. OBJETIVOS ESPECÍFICOS.....8
4. ARQUITECTURA DE SOFTWARE.....9
5. DIAGRAMA DE CASOS DE USO UML.....14
6. DIAGRAMA DE CLASES UML.....20
7. MODELO RELACIONAL.....24
8. INTEGRACIÓN DE LOS MODELOS.....31
9. REQUERIMIENTOS FUNCIONALES.....35
10. REQUERIMIENTOS NO FUNCIONALES.....37
11. MARCO CONCEPTUAL.....39
12. MARCO CONTEXTUAL.....43
13. MARCO INTEGRADOR: INCORPORACIÓN DEL SERVICIO DE INTELIGENCIA ARTIFICIAL.....46
14. PROTOTIPO.....54
CONCLUSIONES.....66
REFERENCIAS.....68

1. INFORMACIÓN GENERAL DEL PROYECTO

En esta sección se retoma la contextualización elaborada en la Fase I y se presenta de manera deliberada sin decisiones técnicas: se describe qué problema existe, qué necesidad genera y qué propiedades debe cumplir la solución, en términos del negocio hotelero. Las decisiones sobre cómo se construirá el sistema, es decir, el estilo arquitectónico, las tecnologías, el motor de base de datos y los mecanismos de control de concurrencia, no hacen parte de la contextualización; se adoptan y se justifican a partir de la sección 4, una vez definidos los objetivos y las funcionalidades. Esta separación atiende la observación recibida sobre la Fase I y permite rastrear cada decisión de diseño hasta un elemento concreto del problema.

1.1 NOMBRE DEL SISTEMA

Hotel Reservas: aplicación móvil transaccional (APK) destinada a la administración de reservas de habitaciones en un hotel.

1.2 DESCRIPCIÓN GENERAL

Hotel Reservas consiste en un sistema de procesamiento de transacciones en línea concebido para utilizarse desde dispositivos móviles. La herramienta permitirá al personal de hoteles pequeños y medianos registrar huéspedes y habitaciones, verificar en tiempo real cuáles se encuentran disponibles, crear, actualizar o cancelar reservas, ingresar pagos, emitir comprobantes, gestionar el ingreso y la salida de los clientes y consultar informes de ocupación. La operación fundamental es la confirmación de una reserva, que debe comportarse como una transacción con propiedades ACID sobre un recurso compartido: la disponibilidad de la habitación. De este modo, dos solicitudes simultáneas para el mismo alojamiento y periodo se resolverán de manera determinista, evitando que se produzcan reservas duplicadas (Elmasri y Navathe, 2016). Esta es una propiedad exigida a la solución y no una decisión de implementación; la manera de garantizarla se define en las secciones 4 y 7.

1.3 PROBLEMÁTICA O NECESIDAD IDENTIFICADA

Durante la Fase I se determinó que muchos hoteles pequeños y medianos administran sus reservas por medios manuales o parcialmente automatizados, entre ellos cuadernos de recepción, archivos compartidos, llamadas telefónicas y aplicaciones de mensajería. Como consecuencia, la información relacionada con cada habitación permanece distribuida entre distintos colaboradores y turnos, sin que exista un repositorio central confiable. Esta dispersión genera cuatro problemas principales: sobreventas o reservas dobles, pérdida de solicitudes, diferencias entre la reserva, el pago y la facturación, y falta de evidencia para atender reclamaciones. Por tanto, se requiere un registro centralizado, seguro y concurrente de la disponibilidad, donde las acciones de confirmar, modificar y cancelar se realicen de forma atómica, aislada y persistente, puedan ejecutarse desde cualquier área del hotel y dejen constancia de cada modificación.

1.4 JUSTIFICACIÓN

La propuesta resulta pertinente porque los inconvenientes identificados producen efectos económicos y reputacionales inmediatos. Una sobreventa puede obligar al hotel a trasladar al cliente o renunciar al ingreso; una reserva perdida puede dejar desocupada una habitación que habría podido venderse; y una inconsistencia en los registros demanda tiempo adicional para conciliar información. La implementación de un sistema transaccional previene reservas duplicadas, concentra la información en una fuente única accesible desde los dispositivos móviles del personal, conserva el historial de cambios y disminuye las labores manuales. En el ámbito académico, la gestión hotelera constituye un ejemplo claro de procesamiento transaccional, ya que la secuencia de las operaciones y el manejo de la concurrencia repercuten directamente en los resultados económicos. Por ello, el proyecto permite llevar a la práctica los contenidos estudiados en la asignatura.

1.5 ALCANCE DE LA SOLUCIÓN

El diseño contempla un solo hotel, un inventario central de habitaciones, dos perfiles de usuario, recepcionista y administrador, y una habitación vinculada a cada reserva. Cubre el ciclo completo de la reserva, desde la consulta de disponibilidad hasta la salida del huésped, así como los informes básicos de ocupación. Se excluyen la conexión con pasarelas de pago, administradores de canales comerciales y plataformas contables externas; estas integraciones se consideran posibles ampliaciones futuras.

1.6 DELIMITACIÓN DE LAS FASES

La Fase I se limitó a la contextualización: describió la problemática de la gestión fragmentada de la disponibilidad, la necesidad de un registro único y concurrente, la justificación del proyecto y la pregunta problema. Su producto es la comprensión del problema y de las

propiedades que debe tener cualquier solución, sin comprometer todavía una tecnología ni un estilo arquitectónico.

La Fase II corresponde al análisis y el diseño. Abarca la formulación de objetivos, el modelo de casos de uso, el diagrama de clases, el modelo relacional, la arquitectura de software y la relación trazable entre todos ellos. Es en esta fase donde se toman las decisiones de diseño y se justifican una a una frente a la problemática: el estilo arquitectónico en la sección 4, la plataforma móvil, el lenguaje y el motor de base de datos en la sección 4.5, y los mecanismos de control de concurrencia en la sección 7.4. En esta fase no se desarrollarán la base de datos, la API ni la aplicación operativa, pues su implementación se reserva para las etapas de construcción y pruebas.

La Fase III amplía el documento maestro con la especificación de requerimientos funcionales y no funcionales, el marco conceptual, el marco contextual y el marco integrador, en el que se propone el servicio de inteligencia artificial que acompañará al sistema. Esta fase mantiene el nivel de análisis y diseño: precisa qué debe hacer el sistema y bajo qué condiciones, pero no implementa la base de datos, la API, la aplicación ni el servicio de inteligencia artificial, cuya construcción y conexión corresponden a las etapas operativas siguientes.

2. OBJETIVO CENTRAL

Diseñar una aplicación móvil transaccional que asegure la integridad, coherencia y trazabilidad de las reservas de habitaciones en hoteles pequeños o medianos. Las operaciones simultáneas de confirmación, modificación y cancelación deberán procesarse con propiedades ACID sobre una fuente central de disponibilidad, con el propósito de evitar las sobreventas y la pérdida de reservas detectadas en el diagnóstico.

3. OBJETIVOS ESPECÍFICOS

En primer lugar, definir y justificar una arquitectura de software que diferencie la interfaz de usuario, las reglas del negocio, la administración de transacciones, el acceso a la información y la base de datos. Así, el procesamiento transaccional permanecerá en un punto central y sus garantías no dependerán del equipo desde el cual se utilice la aplicación.

En segundo lugar, representar mediante casos de uso las funciones disponibles para el recepcionista y el administrador. El modelo debe comprender el registro de huéspedes y habitaciones, la consulta de disponibilidad, todas las etapas de una reserva, el control de entradas y salidas y la generación de informes de ocupación.

En tercer lugar, establecer la estructura estática de la solución a través de un diagrama de clases que incluya las entidades del negocio, sus características, operaciones y vínculos. Se dará especial relevancia al proceso de confirmación de reservas y al registro de auditoría.

En cuarto lugar, formular un modelo relacional con las tablas, claves, restricciones y cardinalidades que eviten reservas confirmadas superpuestas para una misma habitación y permitan gestionar apropiadamente la concurrencia, sobre el motor de base de datos que se seleccione y justifique en la arquitectura.

En quinto lugar, evidenciar la correspondencia entre el problema, los objetivos, las funciones, los casos de uso, las clases, el esquema relacional y la arquitectura, garantizando que todos los componentes del diseño se fundamenten en los elementos previamente definidos.

4. ARQUITECTURA DE SOFTWARE

A partir de esta sección el documento pasa de la contextualización a las decisiones de diseño. Cada decisión se presenta con su justificación y se vincula con el aspecto de la problemática o con el atributo de calidad que la motiva, de modo que sea posible distinguir lo que el problema exige de lo que el equipo eligió para resolverlo. Las decisiones que se toman aquí son cuatro: el estilo arquitectónico, la plataforma de la interfaz de usuario, la tecnología de la lógica de negocio y el motor de base de datos.

4.1 TIPO DE ARQUITECTURA SELECCIONADO

Se propone una arquitectura cliente-servidor estructurada en cuatro niveles: presentación, lógica de negocio, acceso a datos y almacenamiento. La presentación se ejecutará en el dispositivo móvil, mientras que los demás niveles funcionarán en el servidor. Cada capa se relacionará únicamente con la que se encuentra inmediatamente debajo mediante interfaces establecidas. Esta organización coincide con el patrón descrito por Bass et al. (2021), que limita las dependencias entre módulos a relaciones autorizadas entre capas consecutivas. En la presentación se empleará además el patrón MVVM Modelo, Vista y Modelo de Vista, recomendado por Microsoft (2024b) para el desarrollo con .NET MAUI.

4.2 DIAGRAMA DE LA ARQUITECTURA

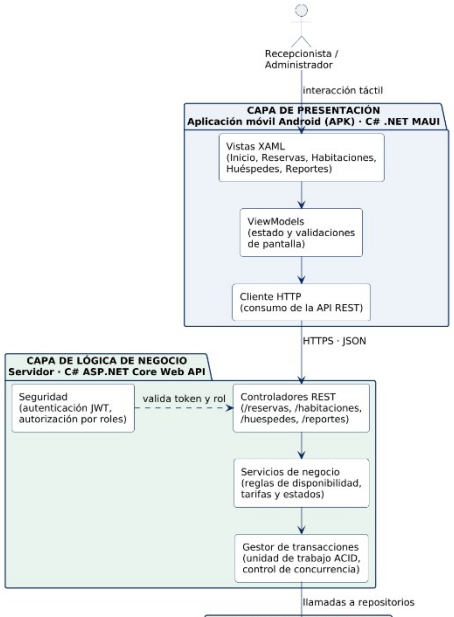


Figura 1 Arquitectura en capas del sistema Hotel Reservas

Nota. Creación propia. Las flechas muestran el sentido de las solicitudes y cada nivel depende exclusivamente del que se encuentra debajo.

4.3 COMPONENTES PRINCIPALES Y FUNCIÓN DE CADA UNO

La capa de presentación corresponde a la aplicación Android programada en C# mediante .NET MAUI y generada como archivo APK. Las vistas XAML representan las pantallas de inicio, reservas, habitaciones, huéspedes e informes disponibles en el repositorio del proyecto. Los modelos de vista administran el estado de las pantallas y validan el formato de la información antes de enviarla, mientras que el cliente HTTP centraliza el acceso a la API REST. Este nivel no implementa reglas del negocio ni se conecta directamente con la base de datos.

La lógica de negocio se implementará mediante una API REST creada en C# con ASP.NET Core Web API. Sus controladores publicarán los recursos de reservas, habitaciones, huéspedes, pagos e informes y transformarán las solicitudes HTTP en invocaciones a los servicios. Estos servicios contendrán las reglas del dominio, como el cálculo de la disponibilidad, la selección de tarifas y las transiciones válidas entre estados. A su vez, el gestor transaccional reunirá en una sola operación acciones como validar nuevamente la disponibilidad, crear la

reserva, registrar el pago y expedir el comprobante, aplicando el control de concurrencia dentro de una única transacción de base de datos. La seguridad validará la identidad mediante tokens JWT y limitará las acciones según el perfil asignado.

Para la capa de acceso a datos se utilizará Entity Framework Core como herramienta de mapeo objeto-relacional. Los repositorios proporcionarán las operaciones de consulta y almacenamiento de cada entidad, y DbContext controlará la correspondencia entre objetos y tablas, la conexión y el alcance de las transacciones. En situaciones que exijan un manejo más específico, como la utilización de sugerencias de bloqueo, se empleará ADO.NET junto con instrucciones T-SQL expresas.

La capa de almacenamiento estará conformada por una base de datos Microsoft SQL Server gestionada desde SQL Server Management Studio. Allí se ubicarán las tablas, claves, reglas de integridad, índices y registros de auditoría. Las transacciones ofrecidas por el motor garantizarán en última instancia el cumplimiento de las propiedades ACID (Microsoft, 2024a).

4.4 COMUNICACIÓN ENTRE COMPONENTES

La aplicación se conectará con la API por medio de solicitudes HTTPS cuyos contenidos se enviarán en JSON, conforme al enfoque REST. Cada recurso dispondrá de una ruta y se operará mediante los métodos GET, POST, PUT y DELETE. Las respuestas incluirán estados HTTP interpretados por la aplicación; por ejemplo, si se intenta asignar una habitación que acaba de ser reservada, se devolverá el código 409 conflicto acompañado de una explicación. En el servidor, los controladores llamarán a los servicios y estos al gestor transaccional mediante métodos directos e inyección de dependencias. El nivel de datos se comunicará con SQL Server a través del proveedor de .NET, que remitirá instrucciones T-SQL usando el protocolo TDS. La

unidad de trabajo comenzará con BEGIN TRANSACTION y terminará con COMMIT o ROLLBACK, según el resultado obtenido.

4.5 JUSTIFICACIÓN DE LA ARQUITECTURA Y DE LAS TECNOLOGÍAS

La organización por capas responde a la necesidad detectada porque sitúa el procesamiento transaccional en el servidor y en la base de datos. En consecuencia, todos los teléfonos consultan y actualizan el mismo registro de disponibilidad, superando la dispersión que ocasiona las reservas dobles. Esta separación también permite modificar o ampliar la interfaz móvil sin cambiar las reglas del negocio, habilitar en el futuro el consumo de la API desde un portal web o un gestor de canales y sustituir o actualizar el motor de datos sin alterar la aplicación. En cuanto a calidad, facilita la modificación y las pruebas aisladas de los componentes, y fortalece la seguridad al centralizar la autenticación y la autorización en un único acceso (Bass et al., 2021). A diferencia de una solución con almacenamiento local, conserva la interacción concurrente entre usuarios, requisito esencial del proyecto.

Las decisiones tecnológicas se justifican de la misma manera. La interfaz se construye como aplicación móvil Android, distribuida en formato APK, porque la necesidad establece que el personal debe consultar y confirmar disponibilidad desde el lugar del hotel donde se encuentre, y porque el docente de la asignatura recomendó esta modalidad; se descartó una base de datos embebida en el dispositivo (SQLite) porque rompería el registro único y, con él, la garantía frente a la doble reserva. Se elige C# con .NET MAUI para la aplicación y C# con ASP.NET Core Web API para la lógica de negocio porque permiten usar un solo lenguaje en las capas de presentación y de servidor, cuentan con soporte del fabricante y se integran de forma nativa con el motor de datos elegido. Se elige Microsoft SQL Server porque ofrece transacciones explícitas, niveles de aislamiento configurables, bloqueos de fila y el tipo ROWVERSION, que

son precisamente los mecanismos que la sección 7.4 emplea para impedir el solapamiento de reservas (Microsoft, 2024a); se descartó MySQL por decisión del equipo y se consideró PostgreSQL como alternativa equivalente, sin ventaja decisiva para este alcance. Ninguna de estas decisiones altera la contextualización de la Fase I; todas responden a ella.

Estado de la implementación en las entregas S10 a S12. Las cuatro capas descritas tienen ya una materialización en el repositorio del proyecto. La capa de presentación es la aplicación .NET MAUI (carpeta HotelReservas, versión 0.3), que incorpora el prototipo funcional de las pantallas transaccionales. Las capas de lógica de negocio y de acceso a datos se implementaron en la API REST HotelReservas.Api, construida con ASP.NET Core 8 y Entity Framework Core: expone los recursos de autenticación (token JWT), disponibilidad, reservas, huéspedes, comprobantes y asistente; la confirmación de reserva ejecuta el procedimiento almacenado con bloqueo y responde con el código 201 cuando la transacción se confirma o 409 cuando otra sesión tomó la habitación, y los cambios de estado aplican concurrencia optimista con la columna ROWVERSION. La capa de almacenamiento son los scripts 01 a 06 de SQL Server. El servicio de inteligencia artificial de la sección 13 se implementó dentro de la API como un componente de la capa de lógica de negocio. La sección 14 relaciona cada recurso de la API con los casos de uso que atiende.

5. DIAGRAMA DE CASOS DE USO UML

5.1 ACTORES

Pressman y Maxim (2021) señalan que un caso de uso explica la manera en que un actor interactúa con el sistema para lograr un propósito; por ello, el modelado comienza con la identificación de quienes participan. La propuesta contempla dos actores humanos. El recepcionista desempeña las tareas operativas: atender clientes, verificar disponibilidad, crear, modificar o cancelar reservas, registrar pagos y controlar el ingreso y la salida. El administrador mantiene el inventario de habitaciones y tarifas, gestiona las cuentas de usuario y consulta los informes de ocupación. Los dos perfiles deben autenticarse antes de acceder a las funciones. No

se incorporan actores externos porque esta fase no considera servicios de pago ni plataformas de terceros.

5.2 DIAGRAMA

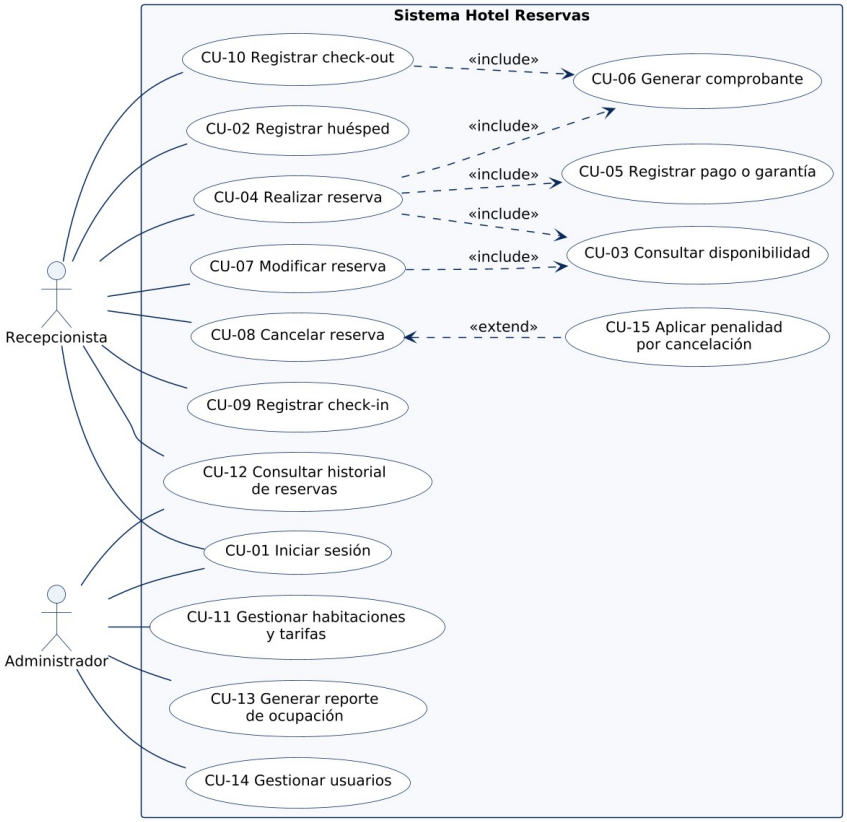


Figura 2 Diagrama de casos de uso del sistema Hotel Reservas

Nota. Diseño propio basado en UML 2.5 (Object Management Group, 2017). El contorno rectangular delimita el sistema; «include» representa una función obligatoria que se reutiliza y «extend» identifica un comportamiento opcional condicionado.

CU-04 y CU-07 incorporan CU-03, Consultar disponibilidad porque una reserva no puede registrarse ni modificarse sin comprobar previamente que la habitación esté libre durante el periodo seleccionado. Esta consulta también puede ser ejecutada directamente por el recepcionista. CU-04 incluye CU-05 y CU-06, puesto que la confirmación requiere registrar el pago o la garantía y producir el comprobante como parte de una misma transacción. Por otra

parte, CU-15 amplía CU-08 únicamente cuando la cancelación se presenta después del plazo admitido por las políticas del establecimiento.

5.3 DESCRIPCIÓN DE LOS PRINCIPALES CASOS DE USO

Nombre: CU-04 Crear una reserva.

Actor principal: Recepcionista.

Objetivo: Formalizar la reserva de una habitación para un huésped durante unas fechas determinadas, sin permitir que el mismo espacio sea asignado a otra solicitud.

Precondiciones: El recepcionista se encuentra autenticado. El huésped ya está registrado o se incorpora durante el proceso mediante CU-02. Debe existir, como mínimo, una habitación perteneciente a la categoría requerida.

Flujo principal: El recepcionista especifica la categoría de habitación, las fechas de llegada y salida y la cantidad de personas. La plataforma ejecuta CU-03, presenta las alternativas libres y muestra sus tarifas. Después de elegir una habitación y vincular al huésped, el sistema abre la transacción, vuelve a comprobar la disponibilidad con bloqueo, genera la reserva con estado confirmada, registra el pago o la garantía por medio de CU-05, produce el comprobante en CU-06, actualiza la bitácora y confirma la operación. Finalmente, informa el código asignado y presenta el comprobante.

Flujo alternativo: Cuando la segunda validación detecta que otra transacción ya ocupó la habitación, todas las acciones se revierten y se informa “habitación no disponible”, sin almacenar datos incompletos.

Resultado esperado: La reserva queda confirmada, vinculada con el pago y el comprobante, almacenada como una unidad indivisible y disponible inmediatamente para todos los usuarios.

Nombre: CU-03 Consultar disponibilidad.

Actor principal: Recepcionista.

Objetivo: Identificar las habitaciones disponibles de una categoría determinada dentro de un intervalo de fechas.

Precondiciones: El usuario inició sesión, las fechas ingresadas son correctas y la salida se programó después de la entrada.

Flujo principal: El usuario selecciona el tipo de habitación y el periodo. El sistema localiza aquellas unidades que no estén en mantenimiento ni tengan reservas confirmadas o huéspedes alojados en fechas que se superpongan con la consulta. Luego presenta los resultados, la tarifa nocturna aplicable y el costo total aproximado.

Resultado esperado: Se obtiene una relación de habitaciones libres calculada con datos del repositorio central, no con registros almacenados localmente.

Nombre: CU-08 Cancelar reserva.

Actor principal: Recepcionista.

Objetivo: Dejar sin efecto una reserva confirmada y devolver la habitación al inventario disponible.

Precondiciones: La reserva está registrada y conserva el estado confirmado. El recepcionista se encuentra autenticado.

Flujo principal: El recepcionista localiza la reserva mediante su código o los datos del cliente y registra la causa de la cancelación. El sistema revisa la política aplicable y, si corresponde, activa CU-15 para calcular la penalidad. Posteriormente abre una transacción, cambia la reserva a cancelada, registra la devolución completa o parcial del pago, actualiza la bitácora y confirma los cambios. Las fechas asociadas a la habitación quedan nuevamente disponibles.

Resultado esperado: La reserva figura como cancelada, el pago se revierte conforme a la política y la habitación se libera. El historial conserva la identidad del responsable y el momento de la operación.

Nombre: CU-09 Registrar check-in.

Actor principal: Recepcionista.

Objetivo: Confirmar la llegada del cliente y registrar que la habitación se encuentra efectivamente ocupada.

Precondiciones: Hay una reserva confirmada cuya entrada corresponde al día en curso y la habitación aparece con estado reservada.

Flujo principal: El recepcionista consulta la reserva, valida la identidad del huésped y confirma su ingreso. Dentro de una sola transacción, el sistema cambia la reserva al estado en casa, marca la habitación como ocupada y guarda el movimiento en la bitácora.

Resultado esperado: La reserva queda en estado en casa, la habitación se registra como ocupada y se almacena la hora de llegada.

Nombre: CU-13 Generar reporte de ocupación.

Actor principal: Administrador.

Objetivo: Consultar la ocupación efectiva y estimada del establecimiento durante un intervalo determinado.

Precondiciones: El administrador está autenticado y existen reservas en el sistema.

Flujo principal: El administrador define el periodo de análisis. Para cada fecha, el sistema determina cuántas habitaciones están ocupadas, reservadas o disponibles y calcula el porcentaje de ocupación utilizando las reservas confirmadas, en casa y cerradas. Los resultados se visualizan en pantalla.

Resultado esperado: Se genera un informe de ocupación acorde con la situación real de las reservas, sin necesidad de realizar conciliaciones manuales.

6. DIAGRAMA DE CLASES UML

El modelo de clases muestra la organización estática necesaria para ejecutar los casos de uso definidos. Sus entidades representan los conceptos identificados en esos procesos: huésped, habitación, categoría de habitación, tarifa, reserva, pago, comprobante y bitácora. Usuario se plantea como clase abstracta y se especializa en Recepcionista y Administrador; además de representar los perfiles del sistema, permite reconocer al responsable de cada acción. ServicioReservas corresponde a la lógica de negocio y no a una entidad del dominio. Se incorpora porque coordina la operación crítica de confirmación.

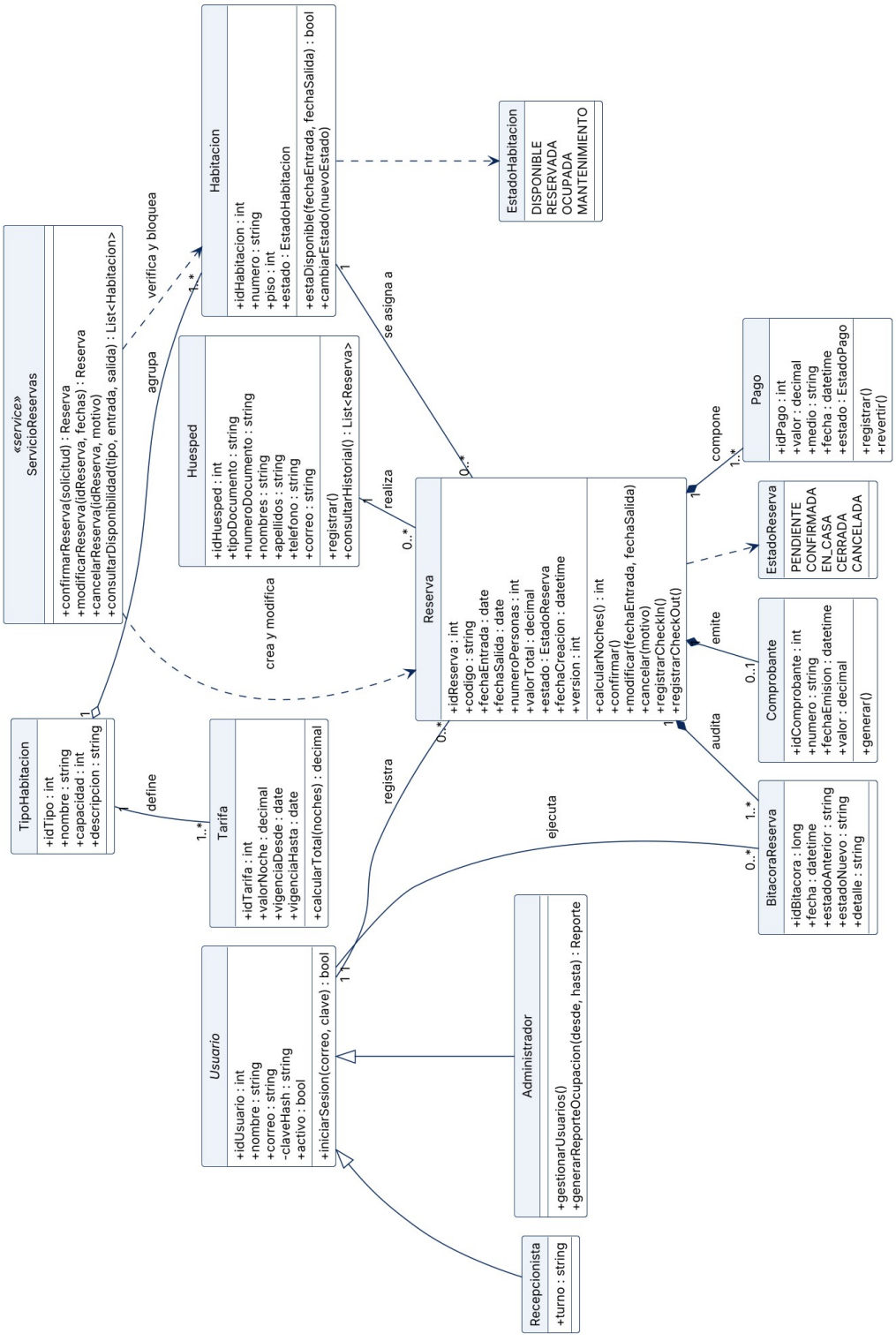


Figura 3 Diagrama de clases del sistema Hotel Reservas

Nota. Elaboración propia con UML 2.5 (Object Management Group, 2017). El diamante sólido expresa composición, el diamante sin relleno representa agregación, las flechas con extremo vacío señalan herencia y los trazos discontinuos indican dependencia.

6.1 RELACIONES Y MULTIPLICIDADES

Cada tipo reúne una o varias habitaciones y dispone de una o más tarifas con periodos definidos. La asociación se representa como agregación, pues las habitaciones pueden existir aunque se reorganice su clasificación. Un huésped puede no tener reservas o registrar varias, mientras que toda reserva debe corresponder a un único huésped. Una habitación admite múltiples reservas en distintos momentos, pero cada reserva se asigna exactamente a una habitación. Cada usuario puede registrar ninguna o muchas reservas, lo que permite establecer quién creó cada registro, y de la misma forma ejecuta ninguna o muchas anotaciones de bitácora, de modo que el responsable de cada cambio de estado queda identificado. Estas multiplicidades coinciden con las cardinalidades del modelo relacional de la sección 7: 1 a 1..N entre tipo y habitaciones y entre tipo y tarifas; 1 a 0..N entre huésped, habitación y usuario respecto de reserva, y entre usuario y bitácora; 1 a 1..N entre reserva y pagos y entre reserva y bitácora; y 1 a 0..1 entre reserva y comprobante.

La composición indica que el pago, el comprobante y la bitácora dependen de la reserva y carecen de sentido sin ella, por lo que desaparecen cuando esta se elimina. Toda reserva contará al menos con un pago la garantía inicial y podrá acumular otros movimientos, como abonos, saldos o devoluciones. También podrá tener un solo comprobante como máximo y deberá incluir al menos una anotación de bitácora correspondiente a su creación. La generalización de Usuario en Recepcionista y Administrador permite reutilizar el mecanismo de autenticación y, al mismo tiempo, distinguir los permisos de cada perfil.

6.2 CORRESPONDENCIA CON LOS CASOS DE USO

CU-04 se materializa mediante confirmarReserva, método de ServicioReservas que consulta estaDisponible en Habitacion, crea la Reserva, registra el Pago, produce el Comprobante y añade la entrada en BitacoraReserva. CU-03 se relaciona con consultarDisponibilidad, operación que revisa las habitaciones de una categoría y aplica estaDisponible. CU-08 se ejecuta con cancelarReserva, que utiliza cancelar en Reserva y revertir en Pago. CU-09 y CU-10 corresponden respectivamente a registrarCheckIn y registrarCheckOut de Reserva, y ambos actualizan Habitacion mediante cambiarEstado. CU-13 se vincula con generarReporteOcupacion de Administrador. Finalmente, el campo version de Reserva contribuye al control de concurrencia optimista detallado en el esquema relacional.

7. MODELO RELACIONAL

El esquema relacional convierte las clases persistentes en tablas de Microsoft SQL Server, transforma sus atributos en columnas con tipos de datos definidos y ubica las claves foráneas en el extremo de multiplicidad “muchos” de cada relación, de acuerdo con las reglas propuestas por Elmasri y Navathe (2016). La jerarquía de Usuario se almacena en una sola tabla denominada USUARIO, acompañada por una columna rol, debido a que sus especializaciones incorporan pocos atributos adicionales. Las enumeraciones se guardan como cadenas breves sujetas a validaciones CHECK. ServicioReservas no se convierte en tabla porque pertenece a la lógica de negocio.

7.1 MODELO ENTIDAD-RELACIÓN (MER)

Antes de presentar el esquema relacional conviene mostrar el modelo entidad-relación (MER), que representa el dominio en términos conceptuales, sin tipos de datos ni claves foráneas, siguiendo la notación propuesta por Chen (1976). La Figura 4 identifica nueve entidades y nueve relaciones. Reserva es la entidad central: recibe participaciones de Huésped (realiza), Habitación (se asigna) y Usuario (registra), todas con cardinalidad 1 a 0..N, y de ella dependen tres entidades débiles, Pago, Comprobante y Bitácora de reserva, cuyas relaciones se dibujan con doble borde porque son identificadoras: ningún pago, comprobante o anotación existe sin su reserva. Tipo de habitación agrupa habitaciones y tarifas con cardinalidad 1 a 1..N, y Usuario ejecuta anotaciones de bitácora, lo que permite identificar al responsable de cada cambio. Los atributos se limitan a los identificadores, subrayados, y a los más representativos de cada entidad.

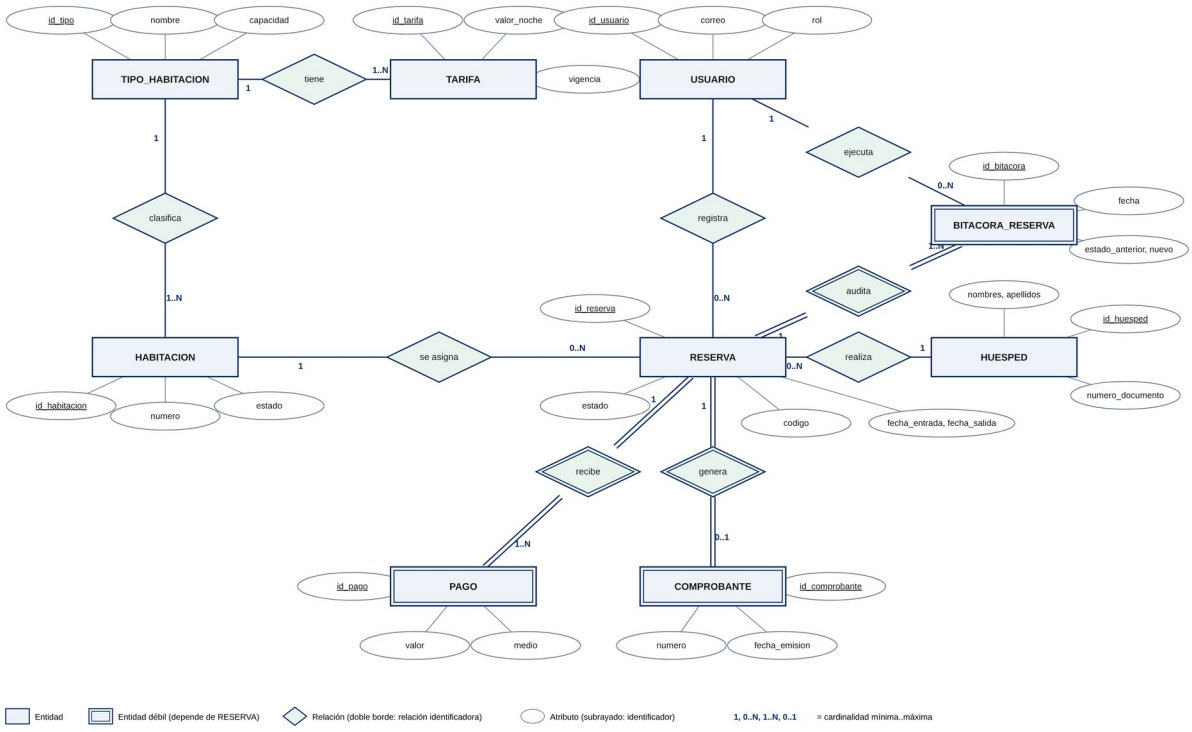


Figura 4 Modelo entidad-relación (MER) del sistema Hotel Reservas

Nota. Elaboración propia con la notación de Chen (1976). Los rectángulos representan entidades, los de doble borde entidades débiles, los rombos relaciones y las elipses atributos; las cardinalidades se expresan como mínimo..máximo. Las mismas entidades y cardinalidades se trasladan al esquema relacional de la Figura 5.

7.2 ESQUEMA RELACIONAL

El esquema relacional de la Figura 5 es la traducción del MER al modelo lógico de Microsoft SQL Server: cada entidad se convierte en una tabla, cada atributo en una columna con tipo de dato, los identificadores en claves primarias y cada relación 1 a N en una clave foránea ubicada en la tabla del lado N. Las entidades débiles conservan la clave foránea obligatoria hacia RESERVA, y la relación 1 a 0..1 con COMPROBANTE se materializa con una restricción UNIQUE sobre id_reserva.

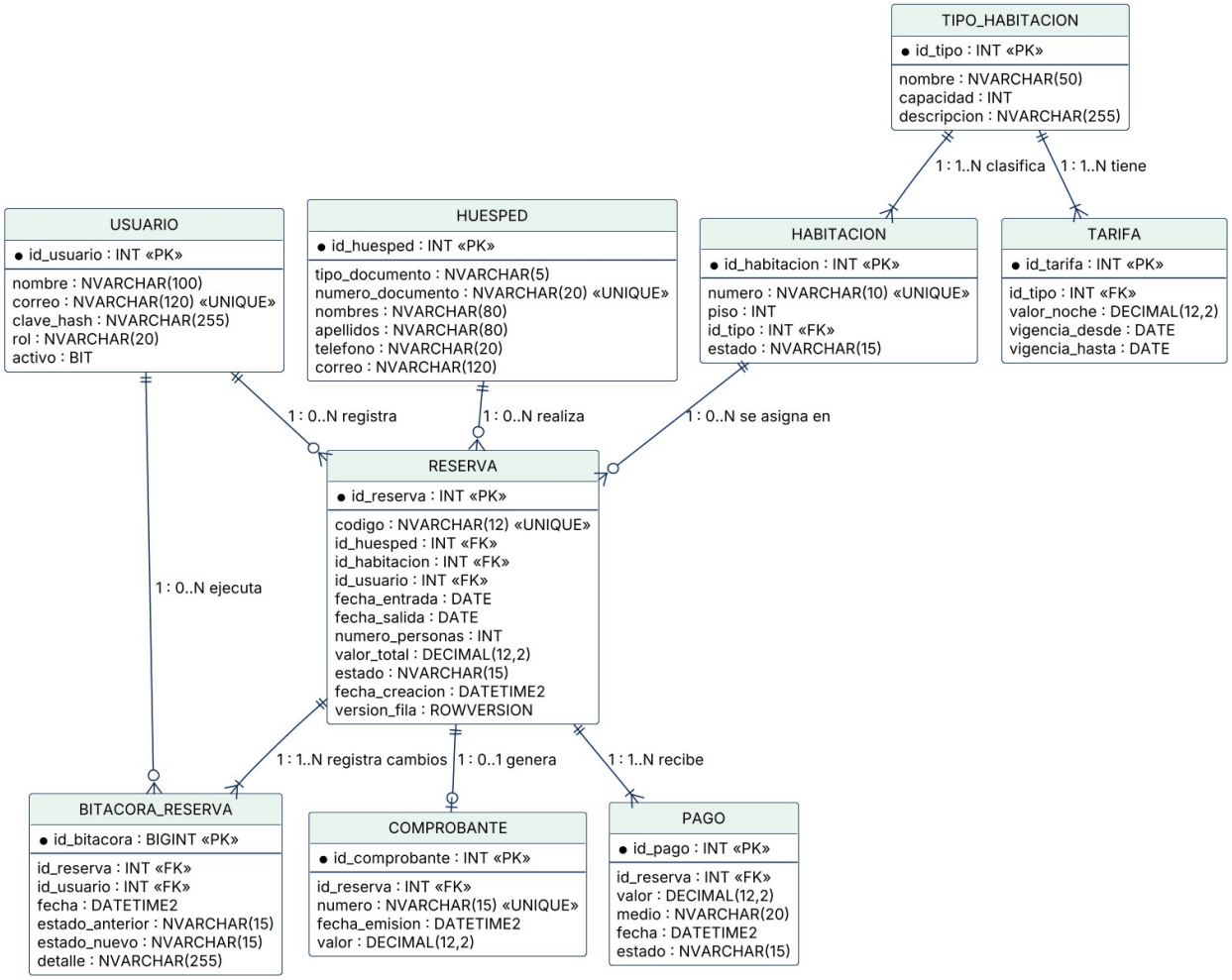


Figura 5 Modelo relacional del sistema Hotel Reservas

Nota. Construcción propia mediante notación de pata de gallo. PK corresponde a clave primaria, FK a clave foránea y UNIQUE a una condición de valor único. Cada etiqueta señala la cardinalidad mínima y máxima de la relación (1 : 0..N, 1 : 1..N o 1 : 0..1), con los mismos valores que las multiplicidades de la Figura 3 y de la sección 6.1.

7.3 DESCRIPCIÓN DE LAS TABLAS

USUARIO conserva las credenciales del personal, su perfil RECEPCIONISTA o ADMINISTRADOR, el correo irrepetible y la contraseña protegida mediante hash. HUESPED reúne los datos de identificación y contacto de los clientes y evita documentos duplicados. TIPO_HABITACION organiza las unidades por denominación y capacidad; TARIFA guarda el precio nocturno de cada categoría y su periodo de aplicación, de manera que se mantenga el historial de valores. HABITACION representa el inventario físico e incluye número único, piso, tipo y situación actual.

RESERVA constituye la tabla principal. Contiene referencias al huésped, la habitación y el usuario responsable, así como las fechas de llegada y salida, la cantidad de ocupantes, el importe calculado con la tarifa vigente, el estado y el momento de creación. SQL Server modifica automáticamente version_fila, columna ROWVERSION utilizada para la concurrencia optimista: el sistema remite la versión previamente consultada y solo permite la actualización cuando todavía coincide con la almacenada. PAGO almacena cada transacción monetaria de la reserva, junto con su forma de pago y condición registrado o revertido. COMPROBANTE conserva el soporte generado al confirmar, identificado con numeración única. BITACORA_RESERVA documenta cada cambio de estado, el usuario que lo efectuó, la fecha y los estados anterior y posterior, aportando la evidencia necesaria frente a posibles reclamaciones.

Ajuste del diseño de datos en la Fase III. Para atender el requerimiento RF16, la tabla RESERVA desglosa el precio en cuatro columnas adicionales: subtotal, que es el producto de las noches por la tarifa vigente; tasa_iva, con el valor 0,19 de la tarifa general del impuesto sobre las ventas o 0 cuando aplica la exención; impuesto, igual al subtotal multiplicado por la tasa; y exento_iva, un indicador que se activa para el huésped extranjero no residente, cuyo servicio de alojamiento está exento del IVA conforme al artículo 481 del Estatuto Tributario (Presidencia de

la República de Colombia, 1989). La columna valor_total pasa a representar el precio neto y una restricción CHECK garantiza que valor_total sea igual a subtotal más impuesto, de manera que el desglose que se muestra al huésped y el que se registra en PAGO y en COMPROBANTE sean siempre coherentes. La tabla HUESPED añade el indicador extranjero_no_residente, del que se toma el valor por defecto de la exención, y el campo telefono, ya existente, es el destino del comprobante compartible por WhatsApp (RF17). La reserva rápida para huésped frecuente (RF18) no requiere columnas nuevas: se calcula sobre las reservas del huésped en estado confirmada, en casa o finalizada. Estos ajustes se implementan en el script 05 del repositorio, que también actualiza el procedimiento de confirmación para calcular el impuesto dentro de la misma transacción.

7.4 RESTRICCIONES QUE GARANTIZAN LA INTEGRIDAD TRANSACCIONAL

Las claves foráneas evitan crear reservas sin huésped, habitación o usuario, y también impiden pagos, comprobantes o movimientos de bitácora desvinculados de una reserva. Mediante restricciones CHECK se valida que la salida sea posterior a la entrada, que la cantidad de personas no exceda la capacidad establecida y que el estado pertenezca al conjunto autorizado. La regla principal, evitar dos reservas activas superpuestas para la misma habitación, se respalda con un índice sobre (id_habitacion, fecha_entrada, fecha_salida) y una comprobación ejecutada dentro de la transacción de confirmación. Para ello se utiliza el aislamiento SERIALIZABLE o las sugerencias UPDLOCK y HOLDLOCK sobre las filas correspondientes, haciendo que dos operaciones simultáneas sobre la misma habitación se atiendan secuencialmente y solo una sea confirmada (Microsoft, 2024a). El fragmento siguiente ejemplifica la transacción prevista para la etapa de construcción.

```
     BEGIN TRANSACTION;
IF EXISTS (SELECT 1 FROM RESERVA WITH (UPDLOCK, HOLDLOCK)
           WHERE id_habitacion = @hab
```

7.5 ARQUITECTURA DE LA BASE DE DATOS

La arquitectura de la base de datos describe cómo se organiza el almacenamiento dentro de la arquitectura en capas de la sección 4. La Figura 6 la resume. La base de datos HotelReservas reside en una instancia de Microsoft SQL Server, en la edición Express o Developer, instalada en el servidor de datos del hotel o en una máquina virtual; ningún componente de la aplicación móvil accede a ella de forma directa, pues toda operación llega a través de la capa de acceso a datos de la API, que se comunica con el motor mediante el proveedor de .NET y el protocolo TDS. Dentro de la base de datos se distinguen cinco elementos: el esquema de nueve tablas derivado del MER, las reglas de integridad declarativas (claves primarias y foráneas, restricciones UNIQUE y CHECK y la columna ROWVERSION de RESERVA), los índices que aceleran la consulta de disponibilidad, la lógica alojada en el motor, representada por el procedimiento almacenado de confirmación y la secuencia que genera los códigos de reserva, y el registro de transacciones con sus copias de seguridad diarias, que garantizan la durabilidad.

Esta organización responde a tres decisiones de diseño de datos. La primera es concentrar las reglas que protegen la integridad en el propio motor, de modo que ningún cliente, ni siquiera uno mal programado, pueda producir dos reservas solapadas o un pago sin reserva. La segunda es separar los permisos: la API se conecta con un inicio de sesión dedicado que solo puede ejecutar consultas y procedimientos sobre la base HotelReservas, mientras que SQL Server Management Studio se reserva para la administración y la ejecución de los scripts de creación, de la transacción y de datos de prueba que acompañan al repositorio del proyecto. La tercera es mantener el esquema pequeño y normalizado, lo que facilita la trazabilidad y permite agregar en

el futuro tablas auxiliares, como la bitácora de consultas del asistente de inteligencia artificial descrita en la sección 13, sin modificar el núcleo transaccional.

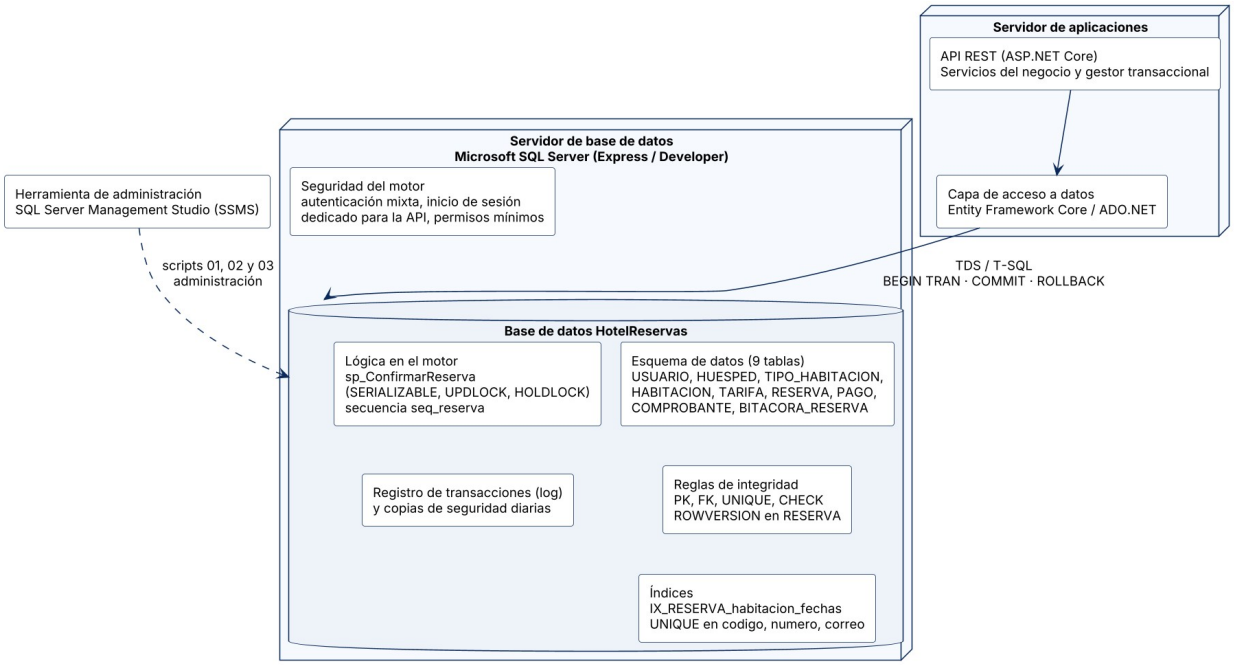


Figura 6 Arquitectura de la base de datos del sistema Hotel Reservas

Nota. Elaboración propia. El servidor de aplicaciones accede a la base de datos únicamente a través de la capa de acceso a datos; SSMS se usa para administración y para ejecutar los scripts del repositorio.

8. INTEGRACIÓN DE LOS MODELOS

La documentación se desarrolló siguiendo una secuencia coherente. La dispersión de los datos y las reservas duplicadas identificadas en el problema condujeron al objetivo general de asegurar integridad, consistencia y trazabilidad en condiciones de concurrencia. Posteriormente, este propósito se dividió en objetivos específicos asociados con la arquitectura, los casos de uso, las clases, el esquema relacional y la trazabilidad. Cada función fue representada por un caso de uso; cada caso se sustenta en determinadas clases; cada clase persistente se refleja en una tabla; y la

arquitectura establece el componente donde debe procesarse cada acción. La Tabla 1 sintetiza estas relaciones para las funciones más importantes.

Tabla 1 Matriz de trazabilidad entre problemática, objetivos, casos de uso, clases, tablas y componentes

Problema	Objetivo	Caso de uso	Clases	Tablas	Componente
Reservas duplicadas por validación sin control	1, 3, 4	CU-03, CU-04	ServicioReservas, Habitacion, Reserva, Tarifa	RESERVA, HABITACION, TARIFA	Gestión transaccional y base de datos mediante aislamiento y bloqueos
Diferencias entre reserva, pago y soporte	3, 4	CU-04, CU-05, CU-06	Reserva, Pago, Comprobante	RESERVA, PAGO, COMPROBANTE	Gestor transaccional como unidad de trabajo indivisible
Pérdida o modificación de reservas sin seguimiento	2, 3	CU-07, CU-08, CU-15	Reserva, Pago, ServicioReservas	RESERVA, PAGO	Servicios del negocio y repositorios
Información desactualizada sobre las habitaciones	2, 3	CU-09, CU-10, CU-11	Habitacion, Reserva, TipoHabitacion	HABITACION, RESERVA, TIPO_HABITACION	Servicios del negocio y DbContext
Ausencia de evidencia para atender reclamaciones	3, 4	CU-04, CU-07, CU-08, CU-09, CU-10	BitacoraReserva, Usuario	BITACORA_RESERVA, USUARIO	Gestor transaccional y seguridad para identificar al usuario

Datos dispersos para apoyar decisiones	2	CU-12, CU-13	Administrador, Reserva, Huesped	RESERVA, HABITACION, HUESPED	Controladores y servicios destinados a reportes
Acceso a los datos sin autorización suficiente	1, 2	CU-01, CU-14	Usuario, Recepcionista, Administrador	USUARIO	Seguridad basada en JWT y perfiles

Nota. Elaboración propia. La numeración de los objetivos específicos sigue el orden presentado en el apartado 3.

La matriz confirma que los componentes fueron concebidos de forma integrada. La confirmación transaccional, respuesta principal ante las reservas dobles, se identifica como CU-04; se coordina desde ServicioReservas usando Reserva, Pago y Comprobante; se guarda en las tablas equivalentes bajo las reglas expuestas en el apartado 7.4; y se procesa en el gestor transaccional de la lógica de negocio con el respaldo del control de concurrencia del motor de datos. De manera semejante, la necesidad de seguimiento se concreta en la clase y la tabla de bitácora, mientras que el módulo de seguridad permite reconocer al usuario responsable de cada movimiento.

Como cierre de la integración conviene precisar qué queda definido en esta fase y qué se traslada a la siguiente. Quedan definidos de manera estable el objetivo central y los objetivos específicos, los dos actores y los quince casos de uso, las clases del dominio con sus multiplicidades, el modelo entidad-relación y las nueve tablas con sus claves y cardinalidades, la arquitectura de la base de datos, el estilo arquitectónico en capas y la trazabilidad entre todos estos elementos. Pasan a la Fase III, que se desarrolla en las secciones 9 a 13, la especificación de los requerimientos funcionales y no funcionales que harán verificable este diseño, la

fundamentación conceptual y contextual, y la propuesta del servicio de inteligencia artificial. La elección final entre bloqueo pesimista y concurrencia optimista, así como los scripts, la API y la aplicación operativa, se reservan para las etapas de construcción y pruebas.

9. REQUERIMIENTOS FUNCIONALES

Los requerimientos funcionales describen las acciones que el sistema transaccional debe permitir a sus usuarios. Siguiendo a Sommerville (2011), se formulan como declaraciones de servicios que el sistema ofrece, con el actor que las solicita y el resultado observable que producen, de manera que cada una pueda convertirse después en un caso de prueba. La Tabla 2 presenta los dieciocho requerimientos funcionales del proyecto. Los quince primeros se corresponden uno a uno con los casos de uso de la sección 5, lo que conserva la trazabilidad construida en la Fase II y evita que aparezcan funciones no respaldadas por el diseño; los tres últimos (RF16 a RF18) se incorporan en esta fase a partir de la revisión del prototipo: el cálculo de impuestos para informar el precio neto y dos funciones diferenciadoras, el comprobante compartible por WhatsApp y la reserva rápida para el huésped frecuente, que simplifican el uso de la aplicación frente a otras soluciones de reservas.

Tabla 2 Requerimientos funcionales del sistema Hotel Reservas

Código	Nombre	Descripción	Actor	Resultado esperado
RF01	Autenticar usuario	Permitir el inicio de sesión con correo y contraseña y asignar las funciones disponibles según el perfil (recepcionista o administrador).	Recepcionista, administrador	Sesión iniciada con un token válido y un menú acorde con el perfil (CU-01).
RF02	Registrar huésped	Capturar tipo y número de documento, nombres, apellidos, teléfono y correo del huésped, impidiendo documentos duplicados.	Recepcionista	Huésped almacenado con identificador único y disponible para asociarse a reservas (CU-02).
RF03	Consultar disponibilidad	Mostrar las habitaciones libres de un tipo para un intervalo de fechas, excluyendo las que estén en mantenimiento o con reservas activas solapadas.	Recepcionista	Lista de habitaciones disponibles con tarifa por noche y valor estimado (CU-03).
RF04	Crear reserva	Registrar una reserva para un huésped, una habitación y un periodo, verificando de nuevo la disponibilidad dentro de la misma transacción.	Recepcionista	Reserva confirmada con código único, o rechazo con el mensaje de habitación no disponible, sin datos parciales (CU-04).

Código	Nombre	Descripción	Actor	Resultado esperado
RF05	Registrar pago o garantía	Asociar a la reserva el valor pagado, el medio de pago y la fecha, como parte de la confirmación o como abono posterior.	Recepcionista	Pago registrado y vinculado a la reserva (CU-05).
RF06	Generar comprobante	Emitir el soporte de la reserva con numeración única, datos del huésped, habitación, fechas y valor.	Recepcionista	Comprobante generado una sola vez por reserva y consultable desde la aplicación (CU-06).
RF07	Modificar reserva	Cambiar fechas, habitación o número de personas de una reserva confirmada, validando la nueva disponibilidad.	Recepcionista	Reserva actualizada y registro del cambio en la bitácora (CU-07).
RF08	Cancelar reserva	Dejar sin efecto una reserva confirmada, registrar el motivo y liberar la habitación para las fechas asociadas.	Recepcionista	Reserva en estado cancelada, pago revertido según la política y habitación liberada (CU-08).
RF09	Registrar check-in	Confirmar la llegada del huésped y marcar la habitación como ocupada.	Recepcionista	Reserva en estado en casa y habitación ocupada (CU-09).
RF10	Registrar check-out	Cerrar la estadía, registrar el saldo final y devolver la habitación al inventario disponible.	Recepcionista	Reserva cerrada y habitación disponible (CU-10).
RF11	Gestionar habitaciones y tarifas	Crear y actualizar tipos de habitación, habitaciones, estados (incluido mantenimiento) y tarifas con vigencia.	Administrador	Inventario y tarifas actualizados y aplicados a las nuevas consultas (CU-11).
RF12	Consultar historial de reservas	Buscar reservas por código, huésped, habitación o fechas y mostrar su estado y movimientos.	Recepcionista, administrador	Listado de reservas con su bitácora de cambios (CU-12).
RF13	Generar reporte de ocupación	Calcular, para un periodo, las habitaciones ocupadas, reservadas y disponibles y el porcentaje de ocupación.	Administrador	Reporte de ocupación coherente con las reservas registradas (CU-13).
RF14	Gestionar usuarios	Crear, activar o desactivar cuentas del personal y asignarles el perfil.	Administrador	Usuarios registrados con perfil y estado (CU-14).
RF15	Aplicar penalidad por cancelación tardía	Calcular y registrar la penalidad cuando la cancelación ocurre fuera del plazo definido por la política del hotel.	Recepcionista	Penalidad calculada, reflejada en el pago y anotada en la bitácora (CU-15).
RF16	Calcular impuestos y precio neto	Desglosar el valor de la reserva en subtotal de alojamiento, IVA del 19 % y precio neto, aplicando la exención del IVA cuando el huésped es extranjero no residente, y registrar el desglose en la reserva, el pago y el comprobante.	Recepcionista	Reserva con subtotal, impuesto y precio neto coherentes (valor_total = subtotal + impuesto); el huésped conoce el valor final antes de confirmar (extiende CU-04 y CU-06).

Código	Nombre	Descripción	Actor	Resultado esperado
RF17	Compartir comprobante por WhatsApp	Generar, a partir del comprobante, un mensaje con el código de reserva, el huésped, la habitación, las fechas y el desglose del precio, y enviarlo al teléfono del huésped por WhatsApp o copiarlo al portapapeles.	Recepcionista	Mensaje enviado al número registrado del huésped en un toque, sin transcribir datos; disponible también desde el historial (extiende CU-06).
RF18	Reserva rápida para huésped frecuente	Reconocer al huésped por su número de documento, mostrar su número de estadías y las preferencias de la última (tipo de habitación, personas, noches y medio de pago) y precargar con ellas la consulta de disponibilidad y la confirmación.	Recepcionista	Huésped con dos o más estadías identificado y reserva precargada en un toque; el recepcionista solo elige la habitación y confirma (extiende CU-03 y CU-04).

Nota. Elaboración propia. El código entre paréntesis en la última columna indica el caso de uso de la sección 5 que materializa cada requerimiento.

Los requerimientos RF03, RF04, RF05 y RF06 forman el núcleo transaccional del sistema: la consulta de disponibilidad alimenta la creación de la reserva, y esta solo se considera completa cuando el pago y el comprobante quedan registrados en la misma unidad de trabajo. Los requerimientos RF07 a RF10 cubren los cambios de estado posteriores y, junto con RF12, dependen de la bitácora para dejar evidencia de cada operación. Los requerimientos RF11 y RF14 corresponden a la administración del inventario y de las cuentas, y RF13 explota los datos registrados para apoyar las decisiones del administrador. Los requerimientos RF16 a RF18 no crean casos de uso nuevos, sino que amplían la transacción de confirmación y el comprobante: RF16 incorpora en la propia transacción el cálculo del IVA y del precio neto, de modo que el valor que ve el huésped es el que queda registrado en RESERVA, PAGO y COMPROBANTE; RF17 reutiliza el comprobante ya emitido para enviarlo por WhatsApp; y RF18 se apoya en el historial de reservas del huésped para precargar la consulta de disponibilidad y la confirmación, sin omitir la verificación de disponibilidad con bloqueo.

10. REQUERIMIENTOS NO FUNCIONALES

Los requerimientos no funcionales establecen las condiciones de calidad bajo las cuales deben prestarse los servicios anteriores. Para organizarlos se toma como referencia el modelo de calidad del producto de la norma ISO/IEC 25010, que agrupa las características en seguridad, eficiencia de desempeño, usabilidad, fiabilidad, mantenibilidad, compatibilidad y portabilidad (ISO/IEC, 2011). A ellas se añade la trazabilidad, que en un sistema transaccional constituye una exigencia del negocio. Siguiendo la indicación del docente, se formulan solo los cinco requerimientos no funcionales que resultan determinantes para este sistema, y cada uno incluye en la Tabla 3 un criterio verificable, de modo que pueda comprobarse en las pruebas de aceptación de las etapas siguientes.

Tabla 3 Requerimientos no funcionales del sistema Hotel Reservas

Código	Atributo	Requerimiento	Criterio verificable
RNF01	Seguridad	Todo acceso a la API debe autenticarse mediante un token JWT emitido al iniciar sesión, y cada operación debe autorizarse según el perfil del usuario. Las contraseñas se almacenan únicamente como hash.	Ninguna solicitud sin token válido obtiene datos; un recepcionista no puede ejecutar RF11 ni RF14; en la base de datos no existe ninguna contraseña en texto claro.
RNF02	Integridad de los datos	La confirmación, modificación y cancelación de reservas deben ejecutarse como transacciones ACID; no pueden existir dos reservas activas solapadas para la misma habitación.	En una prueba con dos sesiones simultáneas sobre la última habitación, exactamente una reserva queda confirmada y la otra recibe el código HTTP 409; tras cualquier fallo no quedan pagos ni comprobantes sin reserva.
RNF03	Rendimiento	Las consultas de disponibilidad y la confirmación de reservas deben responder en tiempos adecuados para la atención en recepción.	Con 25 habitaciones y hasta 10 usuarios concurrentes, la consulta de disponibilidad responde en menos de 2 segundos y la confirmación en menos de 3 segundos en el 95 % de los casos.
RNF04	Usabilidad	La aplicación móvil debe permitir completar las tareas frecuentes con pocos pasos y mensajes claros, sin capacitación extensa.	Un recepcionista nuevo crea una reserva completa (RF03 a RF06) en un máximo de cinco pantallas y en menos de tres minutos; los mensajes de error indican la causa y la acción sugerida.
RNF05	Trazabilidad	Cada cambio de estado de una reserva debe registrar el usuario responsable, la fecha, el estado anterior y el nuevo.	Para cualquier reserva es posible reconstruir su historial completo desde la bitácora, y ninguna anotación carece de usuario o de fecha.

Nota. Elaboración propia con base en las características de calidad de ISO/IEC 25010 (ISO/IEC, 2011).

Los requerimientos RNF02 y RNF05 responden directamente a la problemática de la Fase I, pues son los que impiden la sobreventa y aportan la evidencia para atender reclamaciones. RNF01 protege los datos personales de los huéspedes, cuya recolección y tratamiento se rigen en Colombia por la Ley 1581 de 2012 (Congreso de la República de Colombia, 2012). RNF03 y RNF04 aseguran que la solución sea útil en la recepción de un hotel pequeño o mediano, donde la atención al cliente no admite esperas ni capacitación extensa. Atributos como disponibilidad, mantenibilidad y compatibilidad quedan cubiertos por las decisiones de arquitectura de la sección 4 y no se formulan como requerimientos independientes.

11. MARCO CONCEPTUAL

Esta sección expone los conceptos que fundamentan el proyecto y los relaciona con las funciones descritas. Se organiza en cuatro grupos: los conceptos propios del dominio hotelero, los conceptos de sistemas transaccionales, los conceptos tecnológicos de la plataforma elegida y, por último, la forma en que todos ellos se aplican al prototipo.

11.1 CONCEPTOS FUNDAMENTALES DEL PROYECTO

La reserva es el compromiso mediante el cual el hotel aparta una habitación para un huésped durante un periodo determinado, a cambio de un pago o de una garantía. En el sistema se representa como la entidad Reserva y pasa por los estados pendiente, confirmada, en casa, cerrada y cancelada, que describen su ciclo de vida desde la solicitud hasta la salida del huésped. La disponibilidad es la condición de una habitación que no está en mantenimiento ni tiene reservas activas cuyas fechas se solapen con el periodo consultado; su cálculo correcto es el

requisito previo de toda reserva. La sobreventa, conocida en el sector como overbooking, se produce cuando se aceptan más reservas que habitaciones existen para una misma fecha; en este proyecto se trata como un error que debe impedirse, no como una práctica comercial.

El inventario de habitaciones agrupa las unidades físicas del hotel por tipo y capacidad, y la tarifa fija el precio por noche de cada tipo con un periodo de vigencia, lo que permite conservar el historial de precios. El comprobante es el soporte documental entregado al huésped al confirmar la reserva y constituye la evidencia principal ante una reclamación. El check-in y el check-out son los registros de llegada y salida que vinculan la reserva con la ocupación efectiva de la habitación. El precio neto es el valor final que paga el huésped: el subtotal del alojamiento más el impuesto sobre las ventas (IVA), que en Colombia grava los servicios hoteleros con la tarifa general del 19 %, salvo cuando se prestan a extranjeros no residentes, caso en el que el servicio está exento (Presidencia de la República de Colombia, 1989). El huésped frecuente es aquel que ha completado dos o más estadías en el hotel y cuyas preferencias, registradas en sus reservas anteriores, permiten agilizar una nueva reserva. Estos conceptos definen el vocabulario con el que se nombraron los casos de uso, las clases y las tablas en las secciones anteriores.

11.2 CONCEPTOS DE SISTEMAS TRANSACCIONALES

Un sistema de procesamiento de transacciones en línea (OLTP) es aquel que registra las operaciones del negocio en el momento en que ocurren, atendiendo muchas solicitudes cortas y concurrentes sobre una base de datos compartida (Silberschatz et al., 2020). Una transacción es una secuencia de operaciones que se ejecuta como una unidad lógica de trabajo. Haerder y Reuter (1983) sintetizaron sus propiedades en el acrónimo ACID: atomicidad, porque la transacción se completa en su totalidad o no deja ningún efecto; consistencia, porque lleva la base de datos de un estado válido a otro estado válido; aislamiento, porque las transacciones

concurrentes no observan los resultados intermedios de las demás; y durabilidad, porque los efectos confirmados sobreviven a fallos posteriores. Gray y Reuter (1993) explican que estas garantías son las que permiten que varios usuarios operen al mismo tiempo sobre los mismos datos sin corromperlos.

La concurrencia es la ejecución simultánea de varias transacciones, y el control de concurrencia es el conjunto de mecanismos que garantiza que el resultado sea equivalente a alguna ejecución en serie. En el enfoque pesimista el motor bloquea los registros que una transacción va a leer o modificar hasta que esta termina; en el enfoque optimista cada registro lleva una marca de versión y la actualización solo se acepta si la versión no cambió desde la lectura (Elmasri y Navathe, 2016). El nivel de aislamiento define cuánto se protege una transacción de los efectos de las demás; el nivel serializable es el más estricto y es el que elimina las anomalías que producirían una doble reserva (Microsoft, 2024f). La integridad referencial, mantenida mediante claves foráneas y restricciones, impide que existan pagos, comprobantes o anotaciones sin una reserva válida. Por último, la bitácora o registro de auditoría conserva la secuencia de cambios con su autor y su momento, lo que hace posible reconstruir lo ocurrido y atender reclamaciones.

11.3 CONCEPTOS TECNOLÓGICOS

.NET MAUI es el marco de Microsoft para construir aplicaciones nativas multiplataforma en C# y XAML a partir de un único código base; en este proyecto se utiliza para generar la aplicación Android distribuida como APK (Microsoft, 2024c). ASP.NET Core Web API permite exponer servicios HTTP que reciben y devuelven datos en formato JSON (Microsoft, 2024e), siguiendo el estilo arquitectónico REST, en el que cada recurso del negocio se identifica con una ruta y se manipula con los métodos estándar del protocolo (Fielding, 2000).

Entity Framework Core es un mapeador objeto-relacional que traduce las clases del dominio en tablas y las operaciones sobre objetos en instrucciones SQL, además de administrar el alcance de las transacciones (Microsoft, 2024d).

Microsoft SQL Server es el sistema de gestión de bases de datos relacional elegido para el almacenamiento; ofrece transacciones explícitas, niveles de aislamiento configurables, bloqueos de fila y el tipo de dato ROWVERSION, que son los instrumentos con los que se materializa el control de concurrencia descrito en la sección 7.2 (Microsoft, 2024a). El token JWT es un formato estándar para transportar, de manera firmada, la identidad y el perfil de un usuario autenticado en cada solicitud a la API. Finalmente, los servicios de inteligencia artificial en la nube, como Azure OpenAI Service, ponen a disposición modelos de lenguaje que interpretan y producen texto en lenguaje natural a través de una API, sin que el equipo deba entrenar ni alojar el modelo (Microsoft, 2024g).

11.4 APLICACIÓN AL PROYECTO

Los conceptos anteriores se combinan en el prototipo de la siguiente manera. El vocabulario del dominio hotelero define las entidades y los estados que el sistema registra. Las propiedades ACID se aplican a la confirmación de reservas, que agrupa en una sola transacción la verificación de disponibilidad, la creación de la reserva, el registro del pago, la emisión del comprobante y la anotación en la bitácora. El control de concurrencia, ya sea mediante bloqueos o mediante versiones, resuelve el caso en que dos recepcionistas intentan asignar la misma habitación. La integridad referencial y las restricciones de la base de datos protegen la coherencia de los registros, y la bitácora cumple el requerimiento de trazabilidad. La arquitectura en capas distribuye estos conceptos entre la aplicación móvil, la API y el motor de datos, y el

servicio de inteligencia artificial, descrito en la sección 13, se apoya en la API para interpretar consultas sin intervenir en la transacción crítica.

12. MARCO CONTEXTUAL

12.1 CONTEXTO DEL PROYECTO

El proyecto se sitúa en los hoteles pequeños y medianos, entendidos como establecimientos con un número reducido de habitaciones, administrados por su propietario o por un equipo pequeño, y con personal de recepción que rota por turnos. En este tipo de organización no suele existir un área de tecnología ni un sistema de gestión hotelera contratado, de modo que la operación diaria se apoya en herramientas de propósito general. El entorno de trabajo es la recepción del hotel, pero también los pasillos, los pisos y las zonas comunes, pues el personal se desplaza constantemente; por ello la solución se concibe como una aplicación móvil. El proyecto se desarrolla en el marco de la asignatura Sistemas Transaccionales del programa de Ingeniería de Software de la Universidad Manuela Beltrán, lo que orienta su énfasis hacia el procesamiento confiable de operaciones concurrentes.

12.2 PROBLEMÁTICA IDENTIFICADA

En la Fase I se estableció que estos hoteles administran las reservas mediante cuadernos, hojas de cálculo compartidas, llamadas y mensajería instantánea. La información queda repartida entre personas y turnos sin una fuente única, lo que origina cuatro situaciones recurrentes: habitaciones reservadas dos veces para las mismas fechas, solicitudes que se pierden en el cambio de turno, diferencias entre lo reservado, lo pagado y lo facturado, y ausencia de evidencia para resolver reclamaciones. El punto crítico es el momento en que dos personas confirman una reserva al mismo tiempo sin un mecanismo que las coordine; cualquiera de las herramientas

mencionadas permite que ambas operaciones se completen y que el conflicto se descubra solo cuando el huésped llega.

12.3 USUARIOS O BENEFICIARIOS

El sistema tiene dos usuarios directos. El recepcionista atiende a los clientes y necesita consultar disponibilidad y confirmar, modificar o cancelar reservas de manera rápida, desde cualquier lugar del hotel y con la seguridad de que lo que ve corresponde a la situación real. El administrador mantiene el inventario de habitaciones, las tarifas y las cuentas del personal, y necesita informes de ocupación confiables para tomar decisiones sobre precios y mantenimiento. Los beneficiarios indirectos son los huéspedes, que reciben una confirmación cierta y un comprobante, y el propietario del hotel, que reduce las pérdidas por sobreventa y por habitaciones no vendidas. Los usuarios directos tienen formación operativa y no técnica, por lo que la interfaz debe ser sencilla y los mensajes claros.

12.4 NECESIDAD QUE SE PRETENDE ATENDER

La necesidad es contar con un registro único, centralizado y concurrente de la disponibilidad de habitaciones, en el que las acciones de confirmar, modificar y cancelar se realicen de forma atómica, aislada y durable, se puedan ejecutar desde dispositivos móviles y dejen constancia de quién hizo cada cambio y cuándo. No se trata únicamente de digitalizar el cuaderno de reservas: la mejora consiste en que el propio sistema impida la doble reserva en lugar de confiar en la atención del personal, y en que el pago y el comprobante queden ligados a la reserva desde el momento de su confirmación.

12.5 DESCRIPCIÓN DEL PRODUCTO O PROTOTIPO

Hotel Reservas es una aplicación móvil Android, distribuida como APK, conectada a una API y a una base de datos central. Su propósito es administrar el ciclo completo de las reservas de un hotel garantizando la integridad y la trazabilidad de la información. La aplicación gestiona cinco grupos de datos: los usuarios del personal con su perfil, los huéspedes, el inventario de habitaciones con sus tipos y tarifas, las reservas con sus pagos y comprobantes, y la bitácora de cambios. Sus funcionalidades principales, enumeradas en la Tabla 2, son la autenticación, el registro de huéspedes, la consulta de disponibilidad, la creación, modificación y cancelación de reservas, el registro de pagos y comprobantes, el check-in y el check-out, la gestión del inventario y de los usuarios, el historial de reservas y el reporte de ocupación. Desde el punto de vista del usuario, el flujo típico consiste en consultar la disponibilidad de un tipo de habitación para unas fechas, seleccionar una habitación, asociar al huésped, revisar el precio neto con su desglose de impuestos, registrar el pago y recibir el comprobante en una sola secuencia de pantallas. Dos funciones distinguen a la aplicación de otras soluciones de reservas: el comprobante se envía al huésped por WhatsApp en un toque, sin transcribir datos, y el huésped frecuente se reconoce por su documento para repetir su última estadía con los filtros y el medio de pago ya cargados. El prototipo funcional de la APK (versión 0.3) contiene estas pantallas; la API REST HotelReservas.Api, construida en las entregas S10 a S12, implementa la lógica de negocio y el acceso a datos que las respaldan, y la conexión definitiva de la aplicación móvil a la API es la actividad siguiente.

12.6 ALCANCE DEL PROYECTO

Durante la etapa académica se desarrollará un sistema para un solo hotel, con un inventario central de habitaciones, dos perfiles de usuario y una habitación por reserva, que cubra el ciclo completo de la reserva y los reportes básicos de ocupación. Los entregables previstos son

la base de datos en SQL Server con sus scripts, la API REST con la transacción de confirmación y el control de concurrencia, la aplicación móvil conectada a la API con los casos de uso priorizados, y la demostración de dos usuarios concurrentes sobre la última habitación disponible. Quedan fuera del alcance la integración con pasarelas de pago, con gestores de canales de venta y con plataformas contables, la reserva en línea por parte del huésped y la operación de varios hoteles. El servicio de inteligencia artificial de la sección 13 se diseñó conceptualmente en la Fase III y se implementó en la API REST en las entregas S10 a S12, limitado a funciones de lectura, de modo que no forma parte del núcleo transaccional ni lo modifica.

13. MARCO INTEGRADOR: INCORPORACIÓN DEL SERVICIO DE INTELIGENCIA ARTIFICIAL

13.1 SERVICIO DE INTELIGENCIA ARTIFICIAL INCORPORADO

Se incorpora un asistente de consultas en lenguaje natural basado en Azure OpenAI Service, el servicio de Microsoft que ofrece acceso, mediante una API, a modelos de lenguaje de gran tamaño como los de la familia GPT-4o (Microsoft, 2024g). La elección se justifica por tres razones: el servicio se integra con la misma plataforma en la que se construyen la API y la base de datos, lo que simplifica la autenticación y el despliegue; dispone de la capacidad de llamada a funciones, que permite al modelo traducir una pregunta en una invocación estructurada a una función definida por el equipo (Microsoft, 2024h); y su política de datos establece que la información enviada no se usa para entrenar los modelos (Microsoft, 2024i), condición necesaria para un sistema que maneja datos de huéspedes.

13.2 FUNCIÓN DENTRO DEL PROYECTO

El asistente atenderá preguntas que hoy exigen varios pasos en la interfaz, formuladas tal como el personal las expresa en la recepción: qué habitaciones dobles hay libres del 10 al 12 de octubre, cuánto costaría una suite por tres noches en diciembre, en qué estado está la reserva de un huésped determinado o cuántas llegadas hay programadas para mañana. El modelo no responde con su propio conocimiento: interpreta la pregunta, identifica la función del sistema que puede responderla y los parámetros necesarios, y la API ejecuta esa función sobre el repositorio central. Con el resultado real, el modelo redacta la respuesta en lenguaje natural. La necesidad que ayuda a resolver es la de agilizar la atención al cliente y reducir los errores de interpretación de fechas y tipos de habitación, sin alterar el control transaccional.

13.3 COMPONENTES DE LA INTEGRACIÓN

La integración del servicio se describe a partir de los cinco componentes que intervienen: el modelo de datos, el modelo de inteligencia artificial, la lógica de integración, la interfaz de usuario y la infraestructura. La Figura 7 los sitúa sobre la arquitectura en capas del prototipo, de modo que puede verse qué elementos ya existen y cuáles se agregan. En síntesis, el servicio se incorpora en la capa de lógica de negocio como un componente adicional de la API, la aplicación móvil solo suma una pantalla de conversación y la base de datos conserva intacto su núcleo transaccional. Los cinco componentes fueron implementados en las entregas S10 a S12 en el proyecto HotelReservas.Api del repositorio; al final de la sección 13.3.5, la Tabla 5 indica el archivo que materializa cada uno.

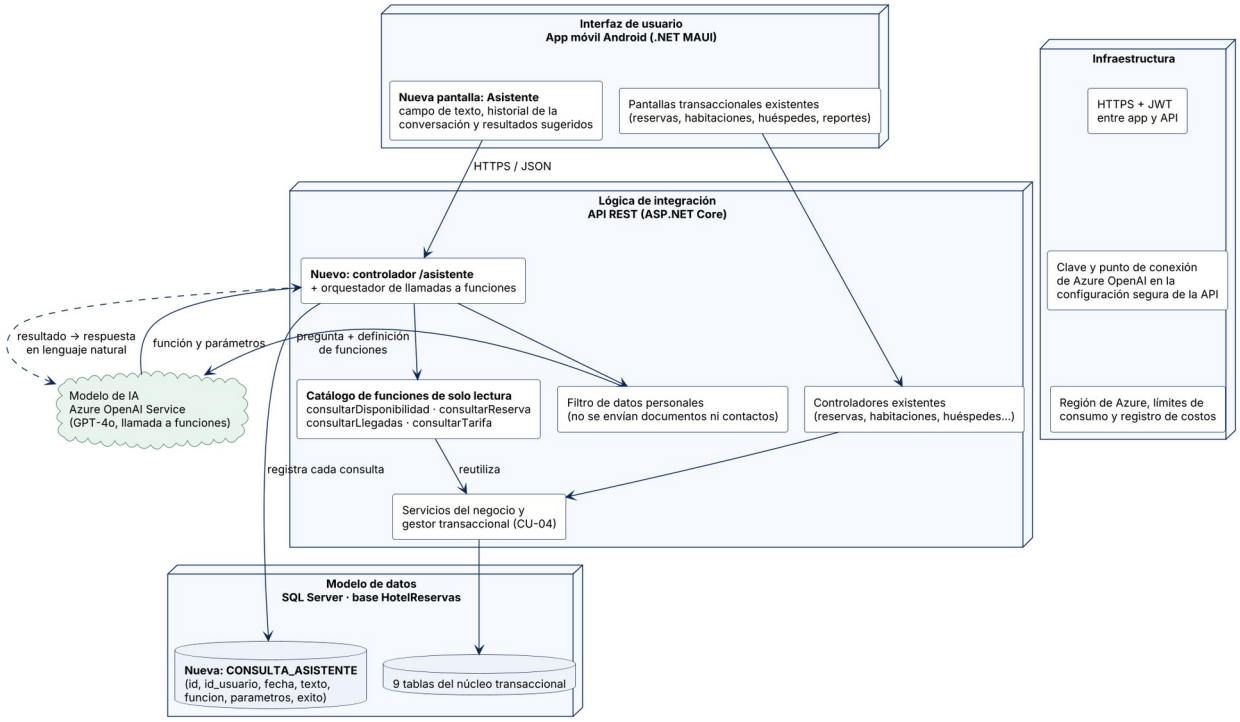


Figura 7 Componentes de la integración del servicio de inteligencia artificial sobre la arquitectura del prototipo

Nota. Elaboración propia. Los elementos marcados como nuevos son los únicos que se agregan a la arquitectura de la Fase II; los demás se reutilizan.

13.3.1 MODELO DE DATOS

El asistente no modifica las nueve tablas del esquema relacional, pues únicamente las consulta a través de las funciones de lectura. El único cambio en el diseño de datos es una tabla auxiliar, CONSULTA_ASISTENTE, que registra cada consulta realizada: identificador, usuario que la hizo, fecha y hora, texto escrito, función invocada, parámetros interpretados y si la respuesta fue exitosa. La tabla se crea con el script 06 del repositorio y registra además el intérprete que atendió la consulta y la duración en milisegundos; una restricción CHECK solo admite los nombres de las cinco funciones del catálogo. Esta tabla cumple dos propósitos: ofrece

trazabilidad del uso del servicio, en coherencia con el requerimiento RNF05, y permite medir el resultado esperado de la sección 13.4. Se relaciona con USUARIO mediante una clave foránea y no tiene relación con RESERVA, porque el asistente nunca crea ni altera reservas. Las funciones de lectura devuelven estructuras derivadas de las tablas HABITACION, TIPO_HABITACION, TARIFA y RESERVA, y en ningún caso incluyen columnas de identificación o contacto de HUESPED.

13.3.2 MODELO DE INTELIGENCIA ARTIFICIAL

El modelo es un modelo de lenguaje de gran tamaño de la familia GPT-4o, consumido como servicio a través de Azure OpenAI Service (Microsoft, 2024g). No se entrena ni se ajusta con datos del hotel; se utiliza en modo de llamada a funciones, en el que la API le entrega la pregunta del usuario junto con la descripción de las funciones disponibles y el modelo responde con el nombre de la función y sus parámetros en formato estructurado (Microsoft, 2024h). Después de que la API ejecuta la función, el modelo recibe el resultado y redacta la respuesta final en español. Un mensaje de sistema fijo le indica que solo puede responder con base en los resultados entregados, que debe pedir aclaración cuando falten fechas o tipo de habitación, y que debe remitir a la pantalla transaccional cualquier solicitud de crear, modificar o cancelar reservas.

En la implementación, la clase InterpreteAzureOpenAI consume el servicio por su interfaz REST de finalización de chat con temperatura cero, en dos llamadas: la primera entrega la pregunta y la descripción de las funciones y recibe la función elegida con sus parámetros; la segunda entrega el resultado real de la función y recibe la respuesta redactada. El punto de conexión, la clave y el nombre del despliegue se leen de la configuración de la API. Como salvaguarda de disponibilidad, cuando no hay credenciales configuradas o el servicio no

responde, la API conmuta de forma automática a un intérprete de reglas (clase InterpreteReglas) que reconoce las mismas consultas mediante expresiones regulares, interpreta las fechas en español (por ejemplo, del 16 al 18 de octubre) y ejecuta el mismo catálogo de funciones; el usuario recibe el mismo tipo de respuesta y la bitácora registra con cuál de los dos intérpretes se atendió cada consulta. Esto permite demostrar el servicio sin conexión a internet y acotar el costo del servicio externo.

13.3.3 LÓGICA DE INTEGRACIÓN

La lógica de integración reside en la API REST, en el backend del prototipo, y se compone de tres piezas. El controlador /asistente recibe la consulta de la aplicación con el token JWT del usuario y la entrega al orquestador. El orquestador aplica el filtro de datos personales, envía la pregunta al modelo, valida que la función devuelta pertenezca al catálogo de funciones de solo lectura (consultarDisponibilidad, consultarTarifa, consultarReserva, consultarLlegadas y consultarReservasConfirmadas), la ejecuta reutilizando los mismos servicios del negocio de los casos de uso CU-03 y CU-12, registra la consulta en CONSULTA_ASISTENTE y devuelve a la aplicación la respuesta redactada junto con los datos estructurados. La Figura 8 muestra la secuencia completa. En el código, estas piezas corresponden a las clases AsistenteController, OrquestadorAsistente, FiltroDatosPersonales y CatalogoFunciones; el catálogo se expone también en el recurso /api/asistente/funciones para que pueda auditarse qué funciones recibe el modelo. Tres reglas delimitan esta lógica: el asistente solo accede a funciones de lectura, de manera que nunca crea, modifica ni cancela reservas; la confirmación (CU-04) se realiza siempre desde la pantalla transaccional habitual, con la verificación y el bloqueo descritos en la sección 7.4; y a la API externa se envían únicamente los datos necesarios para la consulta, sin documentos de identidad ni datos de contacto de los huéspedes.

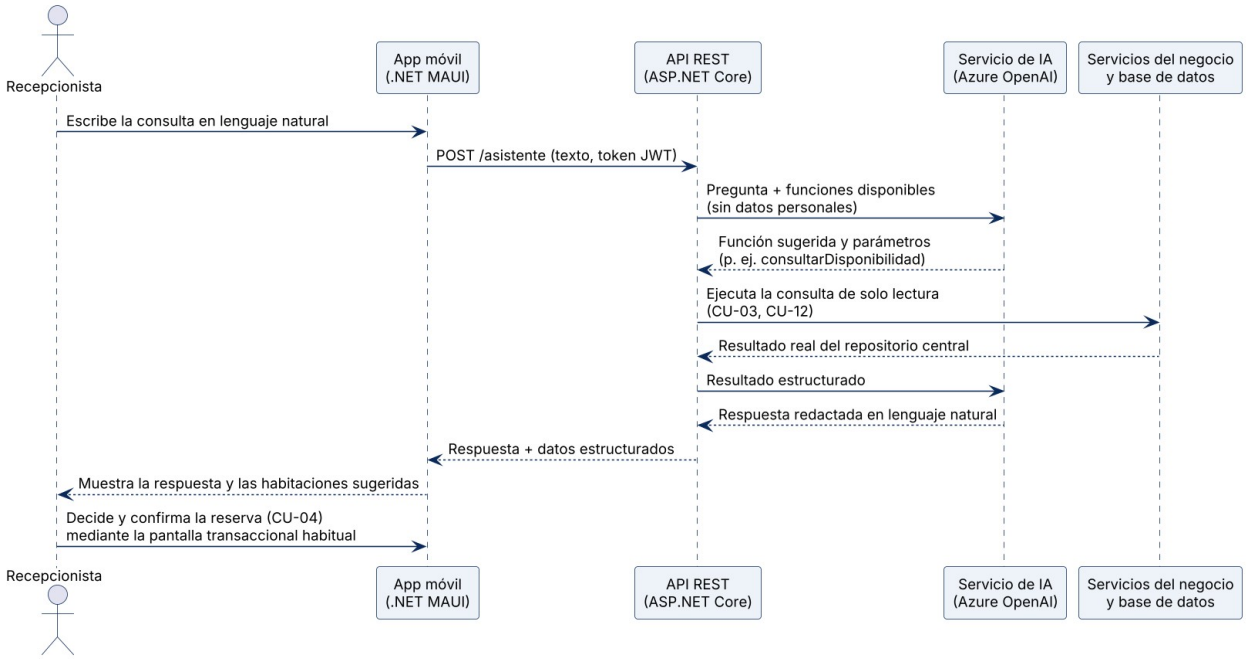


Figura 8 Flujo conceptual del asistente de consultas en lenguaje natural

Nota. Elaboración propia. El servicio de inteligencia artificial interpreta y redacta; la API ejecuta las consultas y conserva el control de las transacciones.

Tabla 4 Ejemplos de consultas y funciones que las atienden

Consulta del usuario	Función de la API invocada	Caso de uso	Resultado
¿Qué habitaciones dobles hay libres del 10 al 12 de octubre?	consultarDisponibilidad(tipo, entrada, salida)	CU-03	Lista de habitaciones disponibles con tarifa y valor estimado.
¿Cuánto costaría una suite por tres noches desde el 20 de diciembre?	consultarDisponibilidad y calcularTotal(noches)	CU-03	Valor estimado según la tarifa vigente del tipo de habitación.
¿En qué estado está la reserva RSV000000045?	consultarReserva(codigo)	CU-12	Estado actual de la reserva y último movimiento de la bitácora.
¿Cuántas llegadas tenemos programadas para mañana?	consultarLlegadas(fecha)	CU-12, CU-13	Número de reservas confirmadas con entrada en la fecha indicada.
Reserva la 305 para Laura Martínez del 3 al 5	No se ejecuta; el asistente indica que la confirmación se realiza en la pantalla de reservas	CU-04 (solo por la interfaz transaccional)	Mensaje de orientación; la transacción conserva su control humano y su bloqueo.

Nota. Elaboración propia. Los nombres de las funciones corresponden a operaciones de ServicioReservas y de los repositorios definidos en la Fase II.

13.3.4 INTERFAZ DE USUARIO

En la aplicación móvil se agrega una pantalla Asistente, accesible desde la barra de pestañas, con un campo de texto, el historial de la conversación y, cuando la respuesta incluye habitaciones, tarjetas con las opciones sugeridas y un botón que conduce a la pantalla de reservas con los datos ya diligenciados. Cada respuesta muestra en letra pequeña la función consultada y la fecha de los datos, para que el recepcionista sepa que la información proviene del repositorio central. La pantalla sigue el patrón MVVM del resto de la aplicación y consume el recurso /api/asistente de la API por HTTPS, enviando la consulta en formato JSON y recibiendo la respuesta redactada, la función consultada, los datos estructurados y el intérprete que la atendió. La sección 14 presenta el prototipo de esta pantalla junto con las pantallas transaccionales.

13.3.5 INFRAESTRUCTURA

La infraestructura necesaria es mínima respecto de la del prototipo. La API conserva su despliegue habitual y agrega, en su configuración segura, la clave y el punto de conexión del recurso de Azure OpenAI, que nunca se incluyen en la aplicación móvil. El recurso se crea en una región de Azure con disponibilidad del modelo, con límites de consumo mensuales y registro de costos, y la comunicación entre la API y el servicio se realiza por HTTPS. La aplicación móvil no requiere cambios de infraestructura, pues sigue hablando únicamente con la API mediante HTTPS y JWT. La base de datos no necesita recursos adicionales, salvo el espacio de la tabla de consultas. En la implementación, los tres valores (punto de conexión, clave y despliegue) se leen de la sección AzureOpenAI del archivo de configuración de la API o de

variables de entorno del servidor, y el archivo publicado en el repositorio los deja vacíos, de modo que el servicio de reglas atiende las consultas hasta que el administrador los configure.

Tabla 5 Correspondencia entre los componentes del servicio de inteligencia artificial y su implementación

Componente	Archivos del repositorio	Qué implementan
Modelo de datos	database/06_consulta_asistente.sql; Data/Entidades.cs y Data/HotelReservasContext.cs	Tabla CONSULTA_ASISTENTE y su mapeo en Entity Framework Core; el asistente solo lee las demás tablas.
Modelo de inteligencia artificial	Asistente/InterpreteAzureOpenAI.cs; Asistente/InterpreteReglas.cs	GPT-4o por Azure OpenAI Service con llamada a funciones y mensaje de sistema fijo; intérprete de reglas de respaldo.
Lógica de integración	Controllers/AsistenteController.cs; Asistente/OrquestadorAsistente.cs; Asistente/CatalogoFunciones.cs	Recurso /api/asistente con JWT, filtro de datos personales, validación contra el catálogo, ejecución y registro.
Interfaz de usuario	HotelReservas/Resources/Raw/ prototipo.html (pantalla Asistente)	Conversación, chips de consultas frecuentes y cita de la función consultada; conexión a la API pendiente.
Infraestructura	HotelReservas.Api/appsettings.json (sección AzureOpenAI); Program.cs	Credenciales en la configuración de la API, HTTPS, tiempo máximo de espera y conmutación automática al respaldo.

Nota. Elaboración propia. Rutas relativas a la carpeta HotelReservas.Api del repositorio, salvo donde se indica otra carpeta.

13.4 RESULTADO ESPERADO

El beneficio esperado para el recepcionista es responder a un cliente en una sola interacción, con datos tomados del repositorio central, en lugar de navegar por varias pantallas y filtros; para el administrador, obtener de manera inmediata cifras que hoy requieren consultar el reporte de ocupación. Se espera además reducir los errores por interpretación de fechas, pues el asistente las normaliza antes de invocar la función. Para el proceso, el resultado es una mejora en la rapidez de atención que no compromete la integridad: el asistente consulta, pero la transacción crítica sigue bajo el control de la API y del motor de datos. El éxito de la propuesta podrá

medirse con el tiempo medio de respuesta a una consulta de disponibilidad y con la proporción de consultas resueltas correctamente en pruebas con usuarios.

13.5 CONSIDERACIONES Y LÍMITES

La propuesta reconoce tres riesgos y la manera de mitigarlos. El primero es que el modelo produzca respuestas no fundamentadas; se mitiga al obligarlo a responder únicamente a partir del resultado de las funciones, sin acceso directo a la base de datos. El segundo es la protección de los datos personales de los huéspedes, atendida con la regla de enviar solo la información indispensable y con la política de no entrenamiento del servicio (Microsoft, 2024i), en coherencia con la Ley 1581 de 2012. El tercero es el costo por consumo del servicio, que se controla limitando el uso al personal autenticado y registrando cada consulta en la bitácora del sistema. En las entregas S10 a S12 el servicio quedó implementado en la API con las tres salvaguardas descritas y con el intérprete de reglas como respaldo; las actividades siguientes son conectar la pantalla móvil al recurso, configurar el recurso de Azure OpenAI y medir con usuarios los indicadores de la sección 13.4.

14. PROTOTIPO

Esta sección presenta el prototipo de la aplicación móvil y lo relaciona con la arquitectura, el backend y el diseño de datos descritos en las secciones anteriores. El prototipo tiene dos componentes: la pantalla de inicio, construida en .NET MAUI y publicada en el repositorio del proyecto en la entrega S7, y el prototipo funcional de las pantallas transaccionales y del asistente, integrado en la APK (versión 0.3) y disponible también como página interactiva en el repositorio, que simula la API y la base de datos con los datos del script 03 y permite ejecutar la transacción de confirmación, la prueba de concurrencia, el cálculo de impuestos, el

comprobante compartible y la reserva rápida. Todas las pantallas comparten la paleta institucional, la barra de pestañas con cinco destinos y la regla de que la aplicación solo muestra datos obtenidos de la API. Dicha API, HotelReservas.Api, forma parte del prototipo desde las entregas S10 a S12: la Tabla 6 relaciona sus recursos con los casos de uso y requerimientos que atienden, de modo que cada pantalla descrita a continuación tiene un recurso del servidor que la respalda.

Tabla 6 Recursos de la API REST HotelReservas.Api y casos de uso que atienden

Recurso	Casos de uso y requerimientos	Función
POST /api/auth/login	CU-01, RF01, RNF01	Valida correo y contraseña (hash PBKDF2) y emite el token JWT con el perfil.
GET /api/disponibilidad	CU-03, RF03, RF16	Habitaciones libres, reservadas y en mantenimiento para un tipo y un intervalo, con subtotal, IVA y precio neto.
POST /api/reservas	CU-04, CU-05, CU-06, RF04, RF05, RF06, RF16, RNF02	Ejecuta sp_ConfirmarReserva; responde 201 con código y comprobante o 409 si la habitación ya no está disponible.
GET /api/reservas, GET /api/reservas/{codigo}	CU-12, RF12, RNF05	Historial y detalle con pagos, comprobante y bitácora.
PUT /api/reservas/{codigo}/checkin, /checkout, /cancelar	CU-08, CU-09, CU-10, RF08 a RF10	Cambios de estado con concurrencia optimista (ROWVERSION); 409 si otra sesión modificó la reserva.
GET y POST /api/huespedes	CU-02, RF02, RF18	Registro de huéspedes y perfil del huésped frecuente con las preferencias de su última estadía.
GET /api/comprobantes/{numero}/compartir	CU-06, RF17	Texto del comprobante y enlace de WhatsApp al teléfono del huésped.
POST /api/asistente	Sección 13	Consulta en lenguaje natural atendida por el servicio de inteligencia artificial (solo lectura).

Nota. Elaboración propia. Todos los recursos, salvo el inicio de sesión, exigen el token JWT; la documentación interactiva (Swagger) y el archivo de pruebas HotelReservas.Api.http acompañan al proyecto en el repositorio.



Figura 9 Pantalla de inicio del prototipo (.NET MAUI, entrega S7)

Nota. Captura de la aplicación Android construida con .NET MAUI, disponible en el repositorio https://github.com/gustavo8015/HotelReservas. Los indicadores de estado de hoy son datos de ejemplo mientras no exista conexión con la API.

La Figura 9 corresponde a la pantalla de inicio: presenta el propósito del sistema, los módulos de reservas, habitaciones, huéspedes y reportes, y las acciones principales. En la arquitectura de la sección 4 pertenece por completo a la capa de presentación; sus tarjetas de

módulo navegan por rutas del Shell y, una vez conectada, el resumen de estado de hoy se obtendrá de la API mediante el recurso de reportes.



Figura 10 Prototipo de la pantalla Consultar disponibilidad (CU-03, RF03)

Nota. Elaboración propia. Captura del prototipo funcional; los datos corresponden a los registros de prueba del script 03 del repositorio y los valores mostrados son precios netos con IVA incluido.

La Figura 10 muestra la consulta de disponibilidad. El recepcionista selecciona tipo, número de personas y fechas; la aplicación envía la solicitud a la API, que ejecuta la consulta sobre HABITACION, TIPO_HABITACION, TARIFA y RESERVA con el índice de la sección 7.5 y devuelve las habitaciones libres, las reservadas en el periodo y las que están en mantenimiento, junto con la tarifa vigente y el valor estimado. El cálculo nunca se hace en el dispositivo, lo que garantiza que todos los usuarios vean el mismo estado del inventario.



Figura 11 Prototipo de la pantalla Nueva reserva, confirmación con desglose de impuestos (CU-04, CU-05 y CU-06; RF04, RF05 y RF16)

Nota. Elaboración propia. Captura del prototipo funcional incluido en la APK; la etiqueta de huésped frecuente y la casilla de exención del IVA corresponden a RF18 y RF16.

La Figura 11 corresponde a la confirmación de la reserva, la transacción crítica del sistema. La pantalla reúne al huésped, la habitación, las fechas y el precio calculado con la tarifa vigente, desglosado en subtotal de alojamiento, IVA del 19 % y precio neto, que es el valor que

se registra como pago; la casilla de huésped extranjero no residente aplica la exención del IVA y recalcula el neto antes de confirmar (RF16). Al pulsar Confirmar reserva, la API ejecuta en el motor de datos el procedimiento de confirmación descrito en la sección 7.4, que vuelve a verificar la disponibilidad con bloqueo y registra en una sola unidad de trabajo la reserva, el pago, el comprobante y la anotación de bitácora. Si otra sesión tomó la habitación, la API responde con el código 409 y la pantalla informa que la habitación ya no está disponible, sin que quede ningún dato parcial. El código de reserva y el número de comprobante que se muestran provienen de la secuencia y de la tabla COMPROBANTE.



Figura 12 Prototipo del comprobante compartible por WhatsApp tras la confirmación (RF06 y RF17)

Nota. Elaboración propia. El texto del comprobante lo genera la API a partir de RESERVA, COMPROBANTE, HUESPED y PAGO; el botón abre WhatsApp con el mensaje dirigido al teléfono registrado del huésped.

La Figura 12 muestra el resultado de la confirmación: el mensaje de éxito con el código de reserva, el número de comprobante y el precio neto, seguido del comprobante compartible. La

aplicación solicita a la API el texto del comprobante, que se construye en el servidor con los datos ya registrados en la transacción (código, huésped, habitación, fechas, subtotal, impuesto, precio neto, medio de pago y estado) y lo entrega a WhatsApp con el número de teléfono del huésped, o lo copia al portapapeles cuando el huésped prefiere otro canal. Esta función responde a la problemática de la Fase I, pues el huésped recibe de inmediato una evidencia escrita de su reserva, y está disponible también desde el historial para cualquier reserva confirmada o en casa.



Figura 13 Prototipo de la reserva rápida para huésped frecuente en la pantalla de inicio (RF18)

Nota. Elaboración propia. El perfil del huésped y sus preferencias provienen de las reservas anteriores registradas en la base de datos.

La Figura 13 presenta la reserva rápida para huésped frecuente. El recepcionista digita únicamente el número de documento; la API consulta HUESPED y las reservas anteriores del huésped, determina si es frecuente (dos o más estadías) y devuelve las preferencias de su última

estadía: tipo de habitación, número de personas, noches y medio de pago. Con el botón Repetir última estadía la aplicación precarga la consulta de disponibilidad y, al elegir la habitación, la pantalla de confirmación ya tiene seleccionados al huésped y su medio de pago, de modo que una reserva habitual se completa en tres toques. La verificación de disponibilidad con bloqueo y la transacción de confirmación son las mismas de la Figura 11: la simplificación está en la captura de datos, no en el control transaccional.



Figura 14 Prototipo de la pantalla Asistente de consultas en lenguaje natural

Nota. Elaboración propia. Cada respuesta indica la función de la API consultada; la última interacción ilustra que el asistente remite la creación de reservas a la pantalla transaccional.

La Figura 14 presenta la pantalla del asistente descrita en la sección 13.3.4. Las preguntas del recepcionista viajan a la API, que las procesa con la lógica de integración de la sección 13.3.3 y responde con texto y datos estructurados; cada respuesta cita la función consultada para que el usuario conozca el origen de la información. La tercera interacción muestra el límite de diseño más importante: ante una solicitud de reservar, el asistente no ejecuta ninguna transacción y orienta al usuario hacia la pantalla de confirmación, de modo que la integridad del núcleo transaccional no depende del modelo de inteligencia artificial. En la versión actual del prototipo incluido en la APK, la pantalla simula localmente estas respuestas con las mismas reglas del intérprete de respaldo; su conexión al recurso /api/asistente de la API, ya implementado, es la actividad siguiente.

CONCLUSIONES

El trabajo de análisis y diseño de esta etapa convirtió la propuesta formulada en la Fase I en una solución organizada. El objetivo general responde directamente al problema y los cinco objetivos específicos se desarrollan dentro del documento. La arquitectura concentra las transacciones en el servidor y la base de datos; el modelo de casos de uso reúne quince funciones distribuidas entre dos actores y sitúa la confirmación de reservas como proceso central; el diagrama de clases estructura esas funciones por medio de las entidades, relaciones y operaciones del dominio; el modelo relacional las traduce en nueve tablas de SQL Server con claves y reglas destinadas a impedir superposiciones; y la matriz demuestra la correspondencia entre las decisiones de diseño y las necesidades detectadas. En síntesis, la prevención de reservas duplicadas queda incorporada a la solución técnica y deja de depender únicamente del cuidado de los colaboradores.

La Fase III completó esa base con los elementos que hacen verificable el diseño. Los quince requerimientos funcionales reproducen uno a uno los casos de uso, los cinco requerimientos no funcionales fijan criterios comprobables de seguridad, integridad, rendimiento, usabilidad y trazabilidad, y los marcos conceptual y contextual explican los fundamentos transaccionales y el entorno hotelero en que se usará la aplicación. El diseño de datos se completó con el modelo entidad-relación y la arquitectura de la base de datos, que concentra en el motor las reglas de integridad. El marco integrador propone un asistente de consultas en lenguaje natural con Azure OpenAI Service, descrito por sus cinco componentes (modelo de datos, modelo de inteligencia artificial, lógica de integración, interfaz de usuario e infraestructura) y limitado a funciones de lectura, de modo que agiliza la atención sin intervenir en la transacción crítica; el prototipo funcional muestra la consulta de disponibilidad, la

confirmación de la reserva con el desglose de impuestos y el precio neto, el comprobante compartible por WhatsApp, la reserva rápida para huésped frecuente y el asistente. En las entregas S10 a S12 se construyó la API REST HotelReservas.Api con la transacción de confirmación sobre el procedimiento almacenado, el control de concurrencia pesimista en la confirmación y optimista en los cambios de estado, los recursos de huésped frecuente y comprobante compartible, y el servicio de inteligencia artificial implementado como componente de la lógica de negocio, con un intérprete de reglas de respaldo que garantiza su funcionamiento sin conexión. Las etapas siguientes deberán conectar la aplicación móvil a la API comenzando por CU-01, CU-03 y CU-04, verificar los criterios de la Tabla 3 mediante pruebas de aceptación, incluida la prueba de dos sesiones concurrentes sobre la API, y configurar el recurso de Azure OpenAI para medir con usuarios el resultado esperado del asistente.

REFERENCIAS

Bass, L., Clements, P. y Kazman, R. (2021). *Software architecture in practice* (4.ª ed.). Addison-Wesley.

Chen, P. P. (1976). The entity-relationship model: Toward a unified view of data. *ACM Transactions on Database Systems, 1*(1), 9-36. https://doi.org/10.1145/320434.320440

Congreso de la República de Colombia. (2012). Ley 1581 de 2012, por la cual se dictan disposiciones generales para la protección de datos personales. *Diario Oficial*, 48587. https://www.funcionpublica.gov.co/eva/gestornormativo/norma.php?i=49981

Elmasri, R. y Navathe, S. B. (2016). *Fundamentals of database systems* (7.ª ed.). Pearson.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Gray, J. y Reuter, A. (1993). *Transaction processing: Concepts and techniques*. Morgan Kaufmann.

Haerder, T. y Reuter, A. (1983). Principles of transaction-oriented database recovery. *ACM Computing Surveys, 15*(4), 287-317. https://doi.org/10.1145/289.291

ISO/IEC. (2011). *ISO/IEC 25010:2011. Systems and software engineering. Systems and software Quality Requirements and Evaluation (SQuaRE). System and software quality models*. International Organization for Standardization. https://www.iso.org/standard/35733.html

Microsoft. (2024a). *Transacciones (Transact-SQL)*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/t-sql/language-elements/transactions-transact-sql

Microsoft. (2024b). *Model-View-ViewModel (MVVM)*. Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/architecture/maui/mvvm

Microsoft. (2024c). *¿Qué es .NET MAUI?* Microsoft Learn. https://learn.microsoft.com/es-es/dotnet/maui/what-is-maui

Microsoft. (2024d). *Información general de Entity Framework Core*. Microsoft Learn. https://learn.microsoft.com/es-es/ef/core/

Microsoft. (2024e). *Creación de API web con ASP.NET Core*. Microsoft Learn. https://learn.microsoft.com/es-es/aspnet/core/web-api/

Microsoft. (2024f). *SET TRANSACTION ISOLATION LEVEL (Transact-SQL)*. Microsoft Learn. https://learn.microsoft.com/es-es/sql/t-sql/statements/set-transaction-isolation-level-transact-sql

Microsoft. (2024g). *¿Qué es Azure OpenAI Service?* Microsoft Learn. https://learn.microsoft.com/es-es/azure/ai-services/openai/overview

Microsoft. (2024h). *Cómo usar la llamada a funciones con Azure OpenAI Service*. Microsoft Learn. https://learn.microsoft.com/es-es/azure/ai-services/openai/how-to/function-calling

Microsoft. (2024i). *Datos, privacidad y seguridad de Azure OpenAI Service*. Microsoft Learn. https://learn.microsoft.com/es-es/legal/cognitive-services/openai/data-privacy

Object Management Group. (2017). *OMG Unified Modeling Language (OMG UML), version 2.5.1*. https://www.omg.org/spec/UML/2.5.1

Pressman, R. S. y Maxim, B. R. (2021). *Ingeniería del software: un enfoque práctico* (9.ª ed.). McGraw-Hill.

Presidencia de la República de Colombia. (1989). Decreto 624 de 1989, por el cual se expide el Estatuto Tributario de los impuestos administrados por la Dirección General de Impuestos Nacionales. *Diario Oficial*, 38756. Artículo 481, bienes y servicios exentos. https://www.secretariasenado.gov.co/senado/basedoc/estatuto_tributario.html

Silberschatz, A., Korth, H. F. y Sudarshan, S. (2020). *Database system concepts* (7.ª ed.). McGraw-Hill.

Sommerville, I. (2011). *Ingeniería de software* (9.ª ed.). Pearson Educación.